

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



ĐỒ ÁN ĐA NGÀNH - HƯỚNG TTNT (CO3107)

Phòng học Thông minh

Giảng viên hướng dẫn:	CN. Tôn Huỳnh Long	
Sinh viên thực hiện:	Nguyễn Tấn Phúc	2312703
(Nhóm 2)	Huỳnh Duy Chương	2310363
	Bùi Nhật Quý	2312864
	Trần Kiến Quốc	2312878
	Đặng Thị Thúy Vi	2313877
	Trần Anh Tài	2252726

Thành phố Hồ Chí Minh, 2026

Danh sách thành viên và nhiệm vụ

Stt	Họ và tên	MSSV	Nhiệm vụ	Hoàn thành
1	Nguyễn Tấn Phúc (Trưởng nhóm)	2312703	- Làm phần AI - Làm phần Backend	100%
2	Huỳnh Duy Chương	2310363	- Làm phần Sensor Node - Hỗ trợ viết báo cáo	100%
3	Bùi Nhật Quý	2312864	- Làm phần AI - Làm phần Backend	100%
4	Trần Kiến Quốc	2312878	- Làm phần AI - Làm phần Database	100%
5	Đặng Thị Thúy Vi	2313877	- Làm phần Frontend - Làm slides	100%
6	Trần Anh Tài	2252726	- Làm phần Gateway - Hỗ trợ viết báo cáo	100%

Bảng 1: Danh sách thành viên và nhiệm vụ

Lời mở đầu

Trong bối cảnh các công nghệ Internet of Things (IoT) và Trí tuệ nhân tạo ngày càng được ứng dụng rộng rãi vào đời sống, việc xây dựng các không gian học tập thông minh trở thành một hướng phát triển thiết thực và có nhiều tiềm năng. Đề tài “Phòng học cá nhân thông minh” được thực hiện với mục tiêu thiết kế một hệ thống hỗ trợ người dùng theo dõi điều kiện môi trường học tập, điều khiển thiết bị và tự động hóa một số thao tác cơ bản nhằm nâng cao sự tiện nghi, hiệu quả và tính cá nhân hóa trong quá trình học tập.

Trong bài làm này, nhóm tập trung phân tích yêu cầu, thiết kế kiến trúc hệ thống và xây dựng các thành phần chính gồm cảm biến, IoT Gateway, Backend, cơ sở dữ liệu, AI service và giao diện Web Dashboard. Hệ thống cho phép thu thập dữ liệu nhiệt độ, độ ẩm, ánh sáng theo thời gian thực, điều khiển đèn/quạt thủ công hoặc bằng giọng nói, đồng thời hỗ trợ thống kê lịch sử hoạt động. Qua đó, đề tài giúp nhóm củng cố kiến thức về phát triển phần mềm đa thành phần, giao tiếp thiết bị IoT và tích hợp mô hình xử lý ngôn ngữ tự nhiên vào một ứng dụng thực tế.

Nhóm xin gửi lời cảm ơn chân thành đến giảng viên hướng dẫn thầy Tôn Huỳnh Long đã tận tình hỗ trợ, định hướng và góp ý trong suốt quá trình thực hiện đề tài. Những nhận xét và hướng dẫn của thầy/cô là cơ sở quan trọng giúp nhóm hoàn thiện nội dung báo cáo cũng như cải thiện chất lượng hệ thống. Do thời gian và kinh nghiệm còn hạn chế, bài làm khó tránh khỏi thiếu sót; nhóm rất mong nhận được thêm ý kiến đóng góp để có thể tiếp tục hoàn thiện đề tài trong tương lai.

Mục lục

Danh sách thành viên và nhiệm vụ	i
Danh mục hình vẽ	v
Danh mục bảng biểu	v
1 Tổng quan đề tài	1
1.1 Đặt vấn đề	1
1.2 Mục tiêu	2
1.3 Phạm vi	3
1.3.1 Phạm vi phần cứng	3
1.3.2 Phạm vi phần mềm và Công nghệ sử dụng	3
1.3.3 Giới hạn của đề tài	4
2 Phân tích hệ thống	5
2.1 Mô tả các chức năng chính	5
2.1.1 Nhóm chức năng Giám sát và Thu thập dữ liệu	5
2.1.2 Nhóm chức năng Điều khiển thiết bị	5
2.1.3 Nhóm chức năng Tự động hóa (Automation)	5
2.1.4 Nhóm chức năng Quản lý và Thống kê	5
2.2 Yêu cầu hệ thống	6
2.2.1 Yêu cầu chức năng	6
2.2.2 Yêu cầu phi chức năng	8
2.3 Danh sách thiết bị	12
3 Thiết kế hệ thống	13
3.1 Use-case diagram	13
3.2 Thiết kế cơ sở dữ liệu (Database)	24
3.2.1 Sơ đồ mối quan hệ thực thể (ERD)	24
3.2.2 Lược đồ ánh xạ	27

3.3	Kiến trúc hệ thống	28
3.3.1	Lớp thiết bị vật lý	29
3.3.2	Lớp IoT Gateway	29
3.3.3	Lớp Server (Backend, Database)	29
3.3.4	Lớp giao diện người dùng (Frontend)	29
3.4	Thiết kế giao diện	30
3.4.1	Màn hình trang chủ	30
3.4.2	Màn hình lịch sử và thống kê	32
4	Hiện thực hệ thống	34
4.1	Mô hình Xử lý ngôn ngữ tự nhiên (NLP)	34
4.1.1	Tập dữ liệu	34
4.1.2	Huấn luyện mô hình	35
4.1.3	Thử nghiệm và đánh giá	36
4.2	Iot Gateway	38
4.2.1	Hàm tự động phát hiện và kết nối thiết bị	39
4.2.2	Xử lý dữ liệu chiều lên (từ Yolo:Bit đến Server)	39
4.2.3	Xử lý dữ liệu chiều xuống (từ Server đến Yolo:Bit)	40
4.2.4	Cơ chế điều phối tác vụ không đồng bộ	40
5	Thiết lập và khởi chạy dự án	42
5.1	Backend	42
5.2	Iot Gateway và Yolo:Bit	43
5.3	AI Service	45
5.4	Front-end	45
6	Tổng kết	46
	Tài liệu tham khảo	48

Danh mục hình vẽ

1	Use Case Diagram của hệ thống phòng học thông minh	13
2	Sơ đồ mối quan hệ thực thể (ERD) của hệ thống	24
3	Lược đồ ánh xạ	27
4	Kiến trúc tổng thể của hệ thống phòng học thông minh	28
5	Giao diện trang chủ ở chế độ sáng	30
6	Giao diện trang chủ ở chế độ tối	31
7	Giao diện lịch sử hoạt động tổng quan	32
8	Giao diện lịch sử lệnh điều khiển chi tiết	33
9	Đánh giá mô hình	37
10	Ma trận nhầm lẫn	37

Danh mục bảng biểu

1	Danh sách thành viên và nhiệm vụ	i
---	--	---

1 Tổng quan đề tài

1.1 Đặt vấn đề

Trong bối cảnh chuyển đổi số và sự bùng nổ Trí tuệ Nhân tạo (AI), các công nghệ như Internet vạn vật (IoT) và Trí tuệ nhân tạo (AI) đang được kết hợp và ứng dụng mạnh mẽ vào mọi mặt của đời sống (AIoT). Tuy nhiên, tại nhà, việc quản lý *không gian học tập cá nhân* và hệ thống thiết bị điện vẫn chủ yếu dựa vào các thao tác thủ công truyền thống.

Điều này dẫn đến hai vấn đề thực tiễn cấp bách: Thứ nhất, tình trạng lãng phí năng lượng diễn ra thường xuyên do người sử dụng quên tắt thiết bị (đèn, quạt) khi rời khỏi phòng. Thứ hai, điều kiện vi khí hậu trong phòng học (nhiệt độ, độ ẩm, cường độ ánh sáng) không được giám sát và điều chỉnh kịp thời, làm giảm sự thoải mái và ảnh hưởng trực tiếp đến hiệu suất học tập, giảng dạy. Xuất phát từ những bất cập nêu trên, nhóm quyết định lựa chọn và thực hiện đề tài "Phòng học cá nhân thông minh". Đề tài hướng tới việc xây dựng một hệ thống giám sát và thu thập dữ liệu môi trường (nhiệt độ, độ sáng, độ ẩm), kết hợp cảm biến sóng âm để nhận diện người dùng. Từ đó, hệ thống có khả năng tự động ra quyết định bật/tắt đèn, quạt khi các chỉ số môi trường vượt quá ngưỡng cài đặt.

Đặc biệt, để nâng cao trải nghiệm người dùng và bắt kịp xu hướng công nghệ, hệ thống còn được tích hợp mô hình AI xử lý ngôn ngữ tự nhiên (NLP) và chuyển đổi giọng nói thành văn bản (Speech-to-text), cho phép người dùng điều khiển thiết bị hoàn toàn bằng khẩu lệnh. Giải pháp này không chỉ giải quyết triệt để bài toán tiết kiệm năng lượng mà còn kiến tạo một không gian học tập tiện nghi, hiện đại và thông minh hóa toàn diện.

1.2 Mục tiêu

Mục tiêu của đề tài là nghiên cứu, thiết kế và xây dựng mô hình phòng học cá nhân thông minh dựa trên sự kết hợp giữa công nghệ Internet of Things (IoT) và Trí tuệ nhân tạo (AI), nhằm giám sát môi trường học tập, hỗ trợ điều khiển thiết bị và tự động hóa các hoạt động trong phòng học. Hệ thống hướng tới việc nâng cao trải nghiệm người dùng, cải thiện điều kiện học tập và tối ưu hóa việc sử dụng năng lượng trong không gian học tập cá nhân. Cụ thể, đề tài tập trung thực hiện các mục tiêu sau:

1. Thiết kế và xây dựng hệ thống thu thập dữ liệu môi trường

Phát triển mô-đun thu thập dữ liệu từ các cảm biến IoT nhằm giám sát các thông số môi trường quan trọng trong phòng học như nhiệt độ, độ ẩm và cường độ ánh sáng theo thời gian thực. Dữ liệu cảm biến được truyền về hệ thống xử lý trung tâm để phục vụ cho các chức năng giám sát, phân tích và điều khiển.

2. Xây dựng cơ chế điều khiển thiết bị trong phòng học

Thiết kế và triển khai hệ thống điều khiển các thiết bị cơ bản trong phòng học như đèn và quạt thông qua giao diện ứng dụng Web. Người dùng có thể thực hiện các thao tác bật/tắt đèn hoặc điều chỉnh mức hoạt động của quạt nhằm đảm bảo điều kiện học tập phù hợp.

3. Tích hợp mô hình trí tuệ nhân tạo cho điều khiển bằng giọng nói

Xây dựng mô-đun xử lý ngôn ngữ sử dụng công nghệ Speech-to-Text để chuyển đổi giọng nói thành văn bản và áp dụng các kỹ thuật xử lý ngôn ngữ tự nhiên (NLP) để phân tích câu lệnh. Hệ thống từ đó có thể xác định thiết bị mục tiêu và hành động điều khiển tương ứng, cho phép người dùng điều khiển phòng học bằng khẩu lệnh.

4. Phát triển cơ chế tự động hóa dựa trên dữ liệu cảm biến

Thiết kế cơ chế tự động vận hành thiết bị dựa trên các ngưỡng môi trường được cấu hình trước. Khi các thông số như nhiệt độ, độ ẩm hoặc ánh sáng vượt quá hoặc thấp hơn ngưỡng cho phép, hệ thống sẽ tự động kích hoạt hoặc điều chỉnh thiết bị nhằm duy trì điều kiện môi trường tối ưu.

5. Xây dựng hệ thống quản lý dữ liệu và trực quan hóa thông tin

Phát triển hệ thống lưu trữ và quản lý dữ liệu cảm biến, đồng thời cung cấp giao diện trực quan hóa dưới dạng biểu đồ thống kê theo thời gian. Chức năng này cho phép người dùng theo dõi sự biến thiên của các thông số môi trường và đánh giá hiệu quả vận hành của hệ thống.

Thông qua các mục tiêu trên, đề tài hướng đến việc xây dựng một hệ thống phòng học thông minh có khả năng giám sát môi trường, điều khiển thiết bị, hỗ trợ tương tác bằng giọng nói và tự động hóa vận hành, đồng thời tạo nền tảng cho việc mở rộng các ứng dụng AIoT trong các không gian học tập và sinh hoạt thông minh trong tương lai.

1.3 Phạm vi

Đề tài được giới hạn trong phạm vi nghiên cứu, thiết kế và thử nghiệm mô hình thu nhỏ của một hệ thống phòng học cá nhân thông minh. Cụ thể, dự án khoanh vùng các giới hạn về phần cứng, phần mềm và công nghệ ứng dụng như sau:

1.3.1 Phạm vi phần cứng

- **Vi điều khiển và Cảm biến:** Sử dụng vi điều khiển YOLO:bit làm thiết bị trạm (End-node). Phần cứng tập trung vào việc thu thập dữ liệu nhiệt độ, độ sáng và độ ẩm.
- **Cơ cấu:** Hệ thống giới hạn ở việc mô phỏng điều khiển bật/tắt đèn, quạt theo mức độ 1, 2, 3 và gửi, hiển thị dữ liệu nhiệt độ, độ ẩm, độ sáng từ các thiết bị phần cứng trên máy tính người dùng.
- **Thiết bị xử lý trung tâm (IoT Gateway):** Sử dụng laptop cá nhân để làm trạm thu phát, đảm nhiệm việc xử lý thu nhận lệnh từ máy người dùng (Backend) và gửi thông tin từ vi điều khiển lên máy người dùng.

1.3.2 Phạm vi phần mềm và Công nghệ sử dụng

- **Trí tuệ nhân tạo (AI):** Trọng tâm của module AI là mô hình nhận diện giọng nói (Speech-2-text) và xử lý ngôn ngữ tự nhiên để bật tắt đèn, quạt.

Mô hình này chạy trên máy tính người dùng để đảm bảo tính thời gian thực.

- **Hệ thống Backend & Cơ sở dữ liệu:** Xây dựng máy chủ cung cấp RESTful API để xử lý logic điều khiển tự động. Cơ sở dữ liệu lưu trữ lịch sử cấu hình, thông số cảm biến và trạng thái hệ thống.
- **Giao diện người dùng (Frontend):** Xây dựng ứng dụng Dashboard Web phục vụ các usecase chính: Điều khiển thiết bị thủ công, điều khiển bằng giọng nói, điều khiển tự động (khi giá trị vượt 1 ngưỡng tự xác định), xem biểu đồ thống kê.
- **Giao thức truyền thông:** Hệ thống được thiết lập luồng xử lý dữ liệu theo mô hình:

```
sensor_node.py <-> gateway.py <-> Backend <-> Frontend
                                     |
                                     +--> ai-service
                                     +--> MySQL
```

1.3.3 Giới hạn của đề tài

- **Về mặt triển khai thực tế:** Hệ thống phần cứng hiện tại chỉ dừng ở mức prototype, chưa can thiệp trực tiếp vào lưới điện dân dụng (220V) nhằm đảm bảo an toàn tối đa trong môi trường thử nghiệm.
- **Về mặt ngôn ngữ AI:** Mô hình xử lý ngôn ngữ tự nhiên được giới hạn huấn luyện (train) trong không gian từ vựng của các câu lệnh điều khiển phòng học cơ bản, chưa mở rộng thành một trợ lý ảo giao tiếp đa miền.

2 Phân tích hệ thống

2.1 Mô tả các chức năng chính

Hệ thống "Phòng học cá nhân thông minh" được thiết kế với các nhóm chức năng toàn diện, bao phủ từ việc thu thập dữ liệu, điều khiển thiết bị đến ứng dụng Trí tuệ nhân tạo. Cụ thể bao gồm 4 nhóm chức năng chính sau:

2.1.1 Nhóm chức năng Giám sát và Thu thập dữ liệu

- **Giám sát môi trường:** Thu thập dữ liệu nhiệt độ, độ sáng và độ ẩm liên tục theo thời gian thực từ các cảm biến đặt trong phòng học.

2.1.2 Nhóm chức năng Điều khiển thiết bị

- **Điều khiển thủ công:** Cho phép người dùng bật/tắt đèn, quạt theo mức độ 1, 2, 3 thông qua giao diện ứng dụng Web.
- **Điều khiển bằng giọng nói:** Tích hợp tính năng nhận diện giọng nói (Speech-to-text) và xử lý ngôn ngữ tự nhiên (NLP). Người dùng có thể ra lệnh bằng giọng nói để điều khiển bật/tắt đèn, quạt một cách tự nhiên mà không cần thao tác vật lý.

2.1.3 Nhóm chức năng Tự động hóa (Automation)

- **Vận hành theo ngưỡng:** Hệ thống có chức năng tự động bật/tắt đèn, quạt khi các chỉ số nhiệt độ, độ sáng, độ ẩm vượt quá ngưỡng đã được cài đặt. (Ví dụ: Tự động bật quạt mức 3 khi nhiệt độ vượt quá một ngưỡng người dùng tự cho, tự động bật đèn khi độ sáng trong phòng xuống thấp).

2.1.4 Nhóm chức năng Quản lý và Thống kê

- **Trực quan hóa dữ liệu:** Cung cấp chức năng xem biểu đồ thống kê về sự biến thiên của nhiệt độ, độ ẩm, độ sáng theo thời gian (giờ, ngày, tuần).

- **Lưu vết hoạt động:** (Có thể bổ sung thêm) Ghi nhận lại lịch sử các thao tác điều khiển thiết bị và các lần hệ thống tự động kích hoạt để quản trị viên dễ dàng theo dõi.

2.2 Yêu cầu hệ thống

Toàn bộ hoạt động của hệ thống sẽ được quy chuẩn thành hai nhóm chính: Yêu cầu chức năng và yêu cầu phi chức năng

2.2.1 Yêu cầu chức năng

1. Chức năng giám sát và thu thập dữ liệu

- **FR-GS01 – Thu thập dữ liệu cảm biến theo thời gian thực:** Hệ thống phải đọc và cập nhật liên tục các giá trị nhiệt độ, độ ẩm và độ sáng từ các cảm biến trong phòng học.
- **FR-GS02 – Hiển thị dữ liệu giám sát:** Hệ thống phải hiển thị tức thời các thông số môi trường hiện tại trên giao diện Web để người dùng theo dõi.
- **FR-GS03 – Đồng bộ dữ liệu về máy chủ:** Dữ liệu cảm biến và trạng thái hiện diện phải được gửi về máy chủ để lưu trữ và phục vụ các chức năng điều khiển, tự động hóa.
- **FR-GS04 – Phát hiện giá trị bất thường:** Hệ thống phải nhận biết khi một thông số môi trường vượt ngưỡng để kích hoạt cơ chế xử lý tương ứng.
- **FR-GS05 – Cung cấp dữ liệu cho thống kê:** Dữ liệu giám sát phải được tổ chức theo mốc thời gian để phục vụ vẽ biểu đồ theo giờ, ngày và tuần.
- **FR-GS06 – Nhận diện trạng thái hiện diện:** Hệ thống sử dụng cảm biến sóng âm để liên tục nhận diện và cập nhật trạng thái có người đang ngồi tại bàn trong phòng học hay không.

2. Chức năng điều khiển thiết bị

- **FR-DK01 – Điều khiển bật/tắt thủ công:** Hệ thống cho phép người dùng thao tác bật hoặc tắt đèn trực tiếp thông qua giao diện ứng dụng Web.

- **FR-DK02 – Điều chỉnh tốc độ quạt thủ công:** Người dùng có thể điều khiển bật/tắt và thay đổi tốc độ quạt theo 3 mức độ (mức 1, 2, 3) trên giao diện Web.
- **FR-DK03 – Tiếp nhận khẩu lệnh giọng nói (STT):** Tích hợp tính năng Speech-to-text để hệ thống có thể thu âm và chuyển đổi câu lệnh bằng giọng nói của người dùng thành văn bản.
- **FR-DK04 – Xử lý ngôn ngữ tự nhiên (NLP):** Hệ thống phân tích câu lệnh văn bản để xác định chính xác ý định điều khiển (bật/tắt) và thiết bị mục tiêu (đèn/quạt/mức độ).
- **FR-DK05 – Thực thi lệnh bằng giọng nói:** Hệ thống tự động kích hoạt điều khiển bật/tắt đèn, quạt dựa trên kết quả phân tích NLP một cách tự nhiên mà người dùng không cần thao tác vật lý.

3. Chức năng tự động hóa (Automation)

- **FR-TD01 – Cài đặt ngưỡng môi trường:** Hệ thống cung cấp chức năng để người dùng tự thiết lập và lưu trữ các giá trị ngưỡng cho nhiệt độ, độ sáng và độ ẩm.
- **FR-TD02 – Tự động vận hành đèn:** Hệ thống tự động ra lệnh bật đèn khi độ sáng trong phòng học xuống thấp hơn ngưỡng đã cài đặt và tắt đèn khi độ sáng vượt ngưỡng.
- **FR-TD03 – Tự động vận hành quạt:** Hệ thống tự động kích hoạt quạt (ví dụ: ở mức 3) khi chỉ số nhiệt độ hoặc độ ẩm vượt quá ngưỡng mà người dùng đã thiết lập.

4. Chức năng quản lý và thống kê

- **FR-QL01 – Trực quan hóa bằng biểu đồ:** Hệ thống cung cấp chức năng hiển thị sự biến thiên của các chỉ số nhiệt độ, độ ẩm, độ sáng dưới dạng biểu đồ thống kê trực quan.
- **FR-QL02 – Phân tích theo mốc thời gian:** Cho phép người dùng tùy chọn lọc và xem biểu đồ dữ liệu thống kê theo các mốc thời gian cụ thể như

giờ, ngày hoặc tuần.

- **FR-QL03 – Lưu vết lịch sử thao tác:** Hệ thống tự động ghi nhận lại toàn bộ lịch sử các thao tác điều khiển thiết bị (bao gồm cả thủ công và giọng nói) cũng như các lần kích hoạt tự động hóa.
- **FR-QL04 – Giao diện theo dõi hoạt động:** Cung cấp màn hình quản lý để người dùng và quản trị viên có thể dễ dàng tra cứu, theo dõi lại các bản ghi lịch sử hoạt động của phòng học.

2.2.2 Yêu cầu phi chức năng

1. Hiệu năng hệ thống (Performance)

- **NFR-PE01 – Thời gian cập nhật dữ liệu cảm biến:** Hệ thống phải cập nhật và hiển thị dữ liệu cảm biến trên giao diện Web với độ trễ trung bình không vượt quá 5 giây kể từ thời điểm cảm biến gửi dữ liệu. Trong hệ thống hiện tại, dữ liệu cảm biến được gửi theo chu kỳ `SEND_INTERVAL = 5s` và giao diện Web cập nhật theo chu kỳ `REFRESH_INTERVALS.REALTIME = 5000 ms`.
- **NFR-PE02 – Thời gian phản hồi lệnh điều khiển:** Các lệnh điều khiển thiết bị từ giao diện Web hoặc từ giọng nói phải được hệ thống xử lý và truyền tới thiết bị IoT với thời gian phản hồi trung bình không vượt quá 3 giây và tối đa không vượt quá 5 giây.
- **NFR-PE03 – Giới hạn tần suất truy cập hệ thống:** Hệ thống phải đảm bảo tần suất truy vấn dữ liệu từ máy chủ và dịch vụ trung gian (Adafruit IO) không vượt quá giới hạn của nền tảng (khoảng 30 yêu cầu/phút) nhằm tránh bị giới hạn truy cập.
- **NFR-PE04 – Quản lý timeout kết nối:** Các yêu cầu HTTP giữa Frontend và Backend phải có thời gian timeout phù hợp (ví dụ: `axios.timeout = 10000 ms`), trong khi các yêu cầu tới dịch vụ Adafruit IO được giới hạn ở `timeout = 5000 ms` để tránh treo hệ thống.
- **NFR-PE05 – Khả năng xử lý đồng thời:** Hệ thống phải có khả năng tiếp nhận và xử lý đồng thời nhiều yêu cầu từ người dùng mà không gây gián

đoạn chức năng giám sát hoặc điều khiển thiết bị.

2. Khả năng mở rộng hệ thống (Scalability)

- **NFR-SC01 – Khả năng mở rộng thiết bị IoT:** Hệ thống phải được thiết kế theo kiến trúc module để có thể dễ dàng tích hợp thêm các cảm biến hoặc thiết bị IoT mới trong tương lai mà không ảnh hưởng đến hệ thống hiện tại.
- **NFR-SC02 – Khả năng mở rộng dữ liệu:** Hệ thống phải hỗ trợ mở rộng cơ chế lưu trữ dữ liệu từ dạng file JSON sang các hệ quản trị cơ sở dữ liệu chuyên dụng như PostgreSQL hoặc các cơ sở dữ liệu chuỗi thời gian (Time-Series Database) như InfluxDB khi quy mô hệ thống tăng lên.
- **NFR-SC03 – Tách biệt các thành phần hệ thống:** Hệ thống cần hỗ trợ việc tách các thành phần Gateway, Backend và Frontend thành các dịch vụ độc lập nhằm thuận lợi cho việc mở rộng và triển khai trên nhiều máy chủ khác nhau.
- **NFR-SC04 – Khả năng mở rộng quy mô thiết bị:** Kiến trúc hệ thống cần cho phép hỗ trợ tối thiểu 100 thiết bị IoT gửi dữ liệu định kỳ mà vẫn đảm bảo hiệu năng ổn định.

3. Bảo mật hệ thống (Security)

- **NFR-SE01 – Xác thực người dùng:** Hệ thống phải yêu cầu người dùng đăng nhập trước khi truy cập các chức năng điều khiển thiết bị hoặc quản lý dữ liệu. Việc xác thực được thực hiện bằng cơ chế JWT với thời gian hết hạn khoảng 7 ngày.
- **NFR-SE02 – Bảo vệ mật khẩu:** Mật khẩu người dùng phải được mã hóa bằng thuật toán băm an toàn (ví dụ: bcrypt với 10 vòng mã hóa) trước khi lưu trữ trong hệ thống.
- **NFR-SE03 – Bảo mật truyền dữ liệu:** Dữ liệu trao đổi giữa các thành phần của hệ thống phải được truyền qua giao thức HTTPS hoặc TLS nhằm đảm bảo tính bảo mật và toàn vẹn dữ liệu.
- **NFR-SE04 – Kiểm soát truy cập API:** Các API nhạy cảm của hệ thống

phải được bảo vệ bằng cơ chế xác thực token và áp dụng giới hạn truy cập (rate limiting) nhằm giảm nguy cơ tấn công từ bên ngoài.

- **NFR-SE05 – Quản lý khóa và thông tin nhạy cảm:** Các khóa bí mật như JWT_SECRET hoặc AIO_KEY phải được lưu trữ trong file cấu hình môi trường (.env) và không được đưa trực tiếp vào mã nguồn.

4. Độ tin cậy và khả năng phục hồi (Reliability)

- **NFR-RE01 – Hoạt động liên tục của hệ thống:** Hệ thống phải đảm bảo khả năng vận hành ổn định trong thời gian dài, hỗ trợ giám sát liên tục các thông số môi trường trong phòng học.
- **NFR-RE02 – Khả năng phục hồi khi lỗi kết nối:** Khi xảy ra lỗi kết nối mạng hoặc lỗi thiết bị, hệ thống phải tự động thử kết nối lại và khôi phục hoạt động mà không cần khởi động lại toàn bộ hệ thống.
- **NFR-RE03 – Cơ chế dự phòng dữ liệu:** Hệ thống phải có cơ chế lưu trữ dữ liệu cục bộ và sử dụng dữ liệu dự phòng trong trường hợp dịch vụ trung gian (như Adafruit IO) không khả dụng.
- **NFR-RE04 – Kiểm tra trạng thái hệ thống:** Hệ thống phải cung cấp các endpoint kiểm tra trạng thái (ví dụ: /health) để xác định tình trạng hoạt động của các thành phần như máy chủ, gateway và dịch vụ lưu trữ.

5. Khả năng sử dụng (Usability)

- **NFR-US01 – Giao diện trực quan:** Giao diện Web phải được thiết kế thân thiện và dễ sử dụng để người dùng có thể nhanh chóng theo dõi thông tin môi trường và điều khiển thiết bị.
- **NFR-US02 – Thiết kế responsive:** Hệ thống phải hỗ trợ hiển thị tốt trên nhiều loại thiết bị khác nhau như máy tính, máy tính bảng và điện thoại thông minh.
- **NFR-US03 – Hỗ trợ điều khiển bằng giọng nói:** Hệ thống phải tích hợp chức năng điều khiển thiết bị bằng giọng nói, sử dụng công nghệ Speech-to-Text để chuyển đổi giọng nói thành văn bản trước khi xử lý.

- **NFR-US04 – Phản hồi tương tác người dùng:** Hệ thống phải cung cấp phản hồi trực quan như thông báo trạng thái, biểu tượng tải dữ liệu (loading) hoặc thông báo lỗi khi người dùng thực hiện thao tác.

6. Tương thích hệ thống (Compatibility)

- **NFR-CP01 – Tương thích trình duyệt:** Hệ thống phải hoạt động ổn định trên các trình duyệt phổ biến như Google Chrome và Microsoft Edge, đồng thời hỗ trợ cơ chế dự phòng cho các trình duyệt khác.
- **NFR-CP02 – Tương thích hệ điều hành:** Gateway của hệ thống phải có khả năng hoạt động trên nhiều hệ điều hành như Windows, macOS và Linux.
- **NFR-CP03 – Tương thích thiết bị phần cứng:** Hệ thống phải hỗ trợ các thiết bị IoT như Yolo:Bit, cảm biến DHT20, cảm biến siêu âm HC-SR04, đèn LED và quạt điều khiển PWM thông qua giao tiếp serial.

7. Khả năng bảo trì và phát triển (Maintainability)

- **NFR-MT01 – Kiến trúc mã nguồn rõ ràng:** Mã nguồn hệ thống phải được tổ chức theo cấu trúc module rõ ràng (services, hooks, components) nhằm giúp việc phát triển và bảo trì thuận tiện hơn.
- **NFR-MT02 – Tiêu chuẩn mã nguồn:** Dự án phải tuân thủ các tiêu chuẩn lập trình và sử dụng các công cụ hỗ trợ như ESLint và Prettier để đảm bảo tính nhất quán của mã nguồn.
- **NFR-MT03 – Tài liệu hệ thống:** Hệ thống phải cung cấp tài liệu hướng dẫn cài đặt, cấu hình và vận hành để hỗ trợ quá trình bảo trì và phát triển trong tương lai.

8. Triển khai và vận hành (Deployment & Operations)

- **NFR-DP01 – Môi trường triển khai:** Hệ thống phải hỗ trợ nhiều môi trường triển khai như phát triển (development), kiểm thử (staging) và vận hành chính thức (production).
- **NFR-DP02 – Tự động hóa triển khai:** Quy trình build và triển khai hệ

thống cần hỗ trợ tích hợp CI/CD để tự động kiểm tra mã nguồn, build và triển khai ứng dụng.

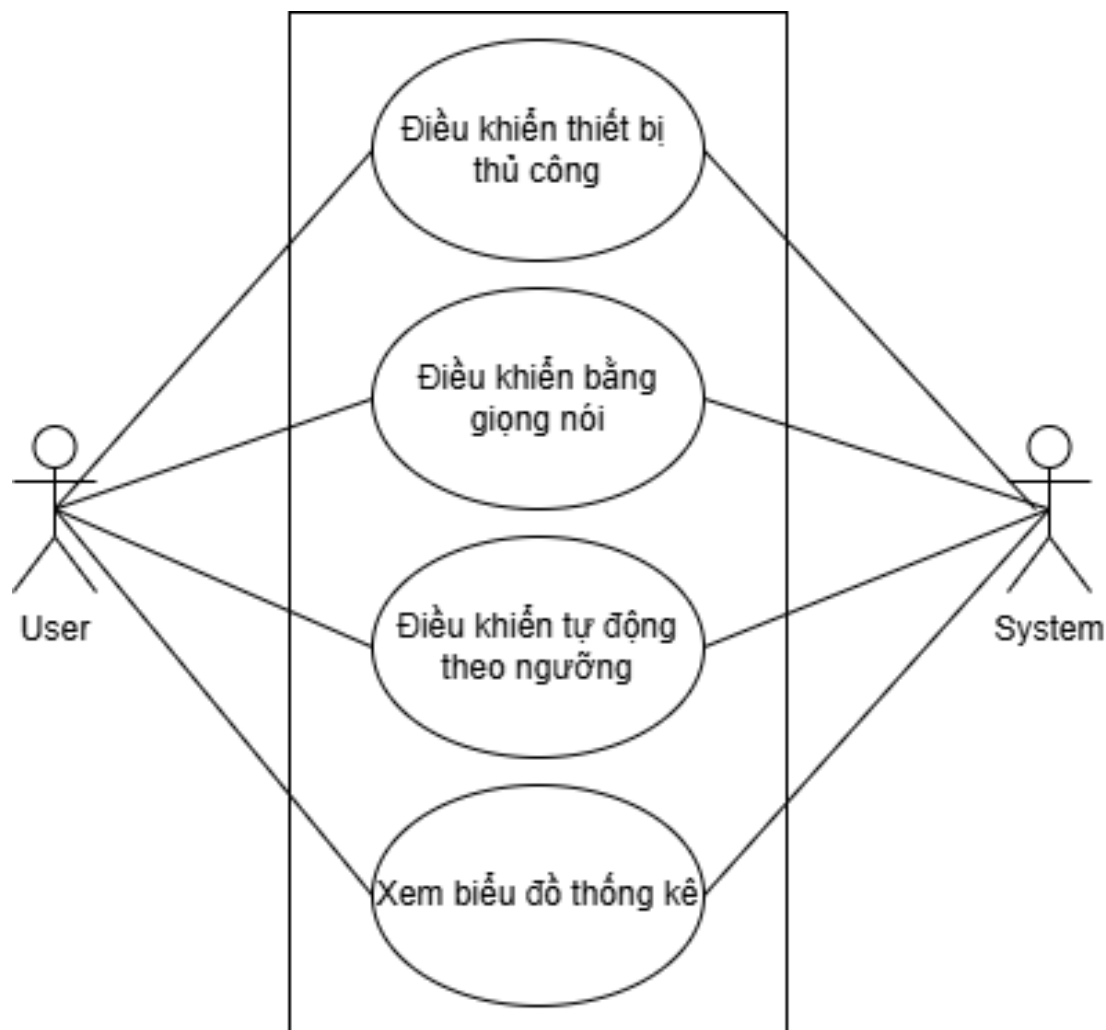
- **NFR-DP03 – Khả năng container hóa:** Các thành phần của hệ thống như Backend và Frontend cần có khả năng đóng gói dưới dạng container (Docker) để thuận tiện cho việc triển khai trên nhiều môi trường khác nhau.
- **NFR-DP04 – Giám sát hệ thống:** Hệ thống cần hỗ trợ cơ chế ghi log và theo dõi trạng thái hoạt động nhằm giúp quản trị viên nhanh chóng phát hiện và xử lý sự cố.

2.3 Danh sách thiết bị

STT	Tên linh kiện	Số lượng
1	Mạch lập trình Yolo:Bit	1
2	Mạch mở rộng	1
3	Module 4 led RGB	1
4	Động cơ quạt mini	1
5	Cảm biến nhiệt độ - độ ẩm DHT20	1
6	Cảm biến ánh sáng	1
7	Dây nối	4

3 Thiết kế hệ thống

3.1 Use-case diagram



Hình 1: Use Case Diagram của hệ thống phòng học thông minh

Use-case name:	Điều khiển thiết bị thủ công
Created by:	Trần Kiến Quốc
Date created:	08/03/2026
Last updated by:	Trần Kiến Quốc

Last updated date:	08/03/2026
Actor:	User, System
Description:	Use case này cho phép người dùng thiết bị và điều khiển thiết bị theo các mức độ 1, 2, 3 thông qua giao diện hệ thống.
Trigger:	Người dùng thực hiện thao tác điều khiển thiết bị trên giao diện.
Pre-conditions:	- Hệ thống đã được khởi động và kết nối với các thiết bị đèn, quạt. - Người dùng truy cập được vào giao diện điều khiển. - Thiết bị đang ở trạng thái sẵn sàng nhận lệnh.
Post-conditions:	- Thiết bị đèn hoặc quạt được cập nhật trạng thái mới theo yêu cầu của người dùng. - Giao diện hiển thị đúng trạng thái hiện tại của thiết bị. - Lịch sử thao tác điều khiển được lưu lại.
Normal flow:	1. User truy cập vào màn hình trang chủ.
	2. System hiển thị các mục trong đó có mục "Điều khiển thiết bị" và các thiết bị được hiển thị ở dưới mục đó.
	3. Đối với từng thiết bị, user chọn bật/tắt cho đèn hoặc chọn mức 1, 2, 3 đối với quạt.
	4. System tiếp nhận và kiểm tra lệnh điều khiển.
	5. System gửi lệnh đến thiết bị tương ứng.
	6. Thiết bị thực thi lệnh.
	7. System cập nhật trạng thái mới lên giao diện.

	8. System lưu lại lịch sử thao tác.
Alternative flow:	- Nếu lưu tên các thiết bị đã bị trùng tên thì sẽ báo lỗi "Tên đã bị trùng vui lòng nhập lại". - Nếu loại thiết bị không được đúng với loại của nó thì hiển thị thông báo "Vui lòng kiểm tra lại loại thiết bị".
Exception:	- Nếu lệnh điều khiển không hợp lệ, System từ chối thực hiện và thông báo lỗi. - Nếu thiết bị không phản hồi, System giữ nguyên trạng thái cũ và thông báo lỗi. - Nếu mất kết nối với bộ điều khiển, System không thể gửi lệnh đến thiết bị
Business rule:	- Đèn chỉ có hai trạng thái: bật hoặc tắt. - Quạt có các mức hợp lệ: tắt, mức 1, mức 2, mức 3. - Mỗi thao tác điều khiển đều phải được ghi nhận vào lịch sử. - Chỉ thực hiện lệnh khi thiết bị đang kết nối và hoạt động bình thường. - Các loại thiết bị phải có tên khác nhau. Ví dụ: Quạt nhà bếp, đèn chính,...

Use-case name:	Điều khiển bằng giọng nói
Created by:	Trần Kiến Quốc
Date created:	08/03/2026
Last updated by:	Trần Kiến Quốc
Last updated date:	08/03/2026

Actor:	User, System
Description:	Use case này cho phép người dùng ra lệnh bằng giọng nói để điều khiển bật/tắt đèn, quạt một cách tự nhiên. Hệ thống sử dụng mô hình Speech-to-Text để chuyển giọng nói thành văn bản, sau đó áp dụng xử lý ngôn ngữ tự nhiên để xác định thiết bị và hành động điều khiển tương ứng.
Trigger:	Người dùng kích hoạt chức năng điều khiển bằng giọng nói và phát ra câu lệnh điều khiển thiết bị.
Pre-conditions:	- Hệ thống đã được khởi động và kết nối với các thiết bị đèn, quạt. - Micro hoặc thiết bị thu âm hoạt động bình thường. - Chức năng Speech-to-Text và NLP đã sẵn sàng xử lý lệnh. - Người dùng đang truy cập giao diện điều khiển của hệ thống.
Post-conditions:	- Thiết bị đèn hoặc quạt được cập nhật trạng thái theo câu lệnh giọng nói hợp lệ. - Trạng thái mới của thiết bị được hiển thị trên giao diện. - Lịch sử lệnh giọng nói và kết quả xử lý được lưu lại trong hệ thống.
Normal flow:	1. User truy cập vào màn hình trang chủ.
	2. System hiển thị các mục trong đó có mục "Điều khiển giọng nói".
	3. System kích hoạt chức năng thu âm.
	4. User phát ra câu lệnh giọng nói, ví dụ: “Bật đèn nhà bếp”, “Tắt quạt số 1”, “Cho quạt số 3 mức 3”.

	5. System ghi nhận tín hiệu âm thanh và chuyển đổi thành văn bản bằng Speech-to-Text.
	6. System phân tích câu lệnh văn bản bằng mô hình NLP để xác định ý định điều khiển và thiết bị mục tiêu.
	7. System kiểm tra tính hợp lệ của câu lệnh.
	8. System gửi lệnh điều khiển đến thiết bị tương ứng.
	9. Thiết bị thực thi lệnh.
	10. System cập nhật trạng thái mới của thiết bị lên giao diện.
	11. System lưu lại lịch sử điều khiển bằng giọng nói.
Alternative flow:	1. User ra lệnh “Bật quạt”, System bật quạt ở mức mặc định đã cấu hình. 2. User ra lệnh “Cho quạt mức 2”, System điều chỉnh quạt sang mức 2. 3. User ra lệnh “Tắt tất cả”, System xử lý theo tập lệnh được hệ thống hỗ trợ nếu có cấu hình trước.
Exception:	- Nếu System không nhận diện được giọng nói, System yêu cầu User phát lệnh lại. - Nếu câu lệnh không thuộc tập từ vựng đã huấn luyện, System thông báo không hiểu yêu cầu. - Nếu micro hoặc thiết bị thu âm không hoạt động, System không thể tiếp nhận câu lệnh. - Nếu thiết bị không phản hồi lệnh điều khiển, System giữ nguyên trạng thái cũ và thông báo lỗi.

Business rule:	- Hệ thống chỉ xử lý các câu lệnh thuộc phạm vi điều khiển phòng học cơ bản. - Câu lệnh giọng nói phải xác định rõ hành động điều khiển và thiết bị mục tiêu. - Quạt chỉ được phép điều chỉnh theo các mức hợp lệ: tắt, mức 1, mức 2, mức 3. - Mọi lệnh giọng nói hợp lệ hoặc không hợp lệ đều có thể được ghi nhận để phục vụ theo dõi và cải thiện hệ thống.
-----------------------	--

Use-case name:	Điều khiển tự động theo ngưỡng
Created by:	Trần Kiến Quốc
Date created:	08/03/2026
Last updated by:	Trần Kiến Quốc
Last updated date:	08/03/2026
Actor:	User, System
Description:	Use case này cho phép hệ thống tự động bật/tắt đèn, quạt khi các giá trị nhiệt độ, độ sáng hoặc độ ẩm vượt quá ngưỡng do người dùng tự thiết lập. Chức năng này giúp phòng học vận hành thông minh mà không cần người dùng can thiệp trực tiếp trong mọi tình huống.
Trigger:	System phát hiện giá trị nhiệt độ, độ sáng hoặc độ ẩm vượt hoặc thấp hơn ngưỡng đã được cấu hình.

Pre-conditions:	- User đã thiết lập và lưu các ngưỡng cho nhiệt độ, độ sáng và độ ẩm. - Hệ thống đang thu thập dữ liệu cảm biến theo thời gian thực. - Chế độ tự động hóa đã được kích hoạt. - Thiết bị đèn, quạt đang ở trạng thái sẵn sàng nhận lệnh.
Post-conditions:	- Đèn hoặc quạt được bật/tắt hoặc điều chỉnh mức phù hợp theo quy tắc ngưỡng. - Trạng thái mới của thiết bị được cập nhật lên giao diện hệ thống. - Sự kiện tự động kích hoạt được lưu vào lịch sử hoạt động.
Normal flow:	1. System liên tục nhận dữ liệu nhiệt độ, độ sáng và độ ẩm từ các cảm biến. 2. System so sánh các giá trị hiện tại với các ngưỡng mà User đã thiết lập và User có thể thấy các ngưỡng và trạng thái cho biết đã bật hay tắt (đối với đèn) hoặc ở mức độ nào (đối với quạt) ở giao diện chính ở mục "Chế độ tự động". 3. Nếu độ sáng thấp hơn ngưỡng cho phép, System xác định cần bật đèn. Nếu nhiệt độ hoặc độ ẩm cao hơn ngưỡng cho phép, System xác định cần bật quạt hoặc tăng mức quạt. 4. System kiểm tra trạng thái hiện tại của thiết bị để tránh gửi lệnh lặp lại không cần thiết. 5. System gửi lệnh điều khiển thích hợp đến thiết bị đèn hoặc quạt.

	6. Thiết bị thực thi lệnh.
	7. System cập nhật trạng thái mới của thiết bị lên giao diện.
	8. System lưu lại lịch sử kích hoạt tự động.
Alternative flow:	1. Nếu độ sáng trở lại mức bình thường, System có thể tắt đèn theo cấu hình đã cài đặt.
	2. Nếu nhiệt độ chỉ vượt ngưỡng nhẹ, System có thể bật quạt ở mức thấp thay vì mức cao nhất.
	3. Nếu thiết bị đã ở đúng trạng thái mong muốn, System không gửi lệnh điều khiển mới.
Exception:	- Nếu dữ liệu cảm biến không hợp lệ hoặc bị lỗi, System không thực hiện điều khiển tự động từ dữ liệu đó. - Nếu User chưa thiết lập ngưỡng, System bỏ qua chức năng tự động hóa. - Nếu thiết bị không phản hồi lệnh, System ghi nhận lỗi và thông báo trên giao diện. - Nếu mất kết nối giữa các thành phần hệ thống, System tạm dừng cơ chế tự động hóa.

Business rule:	- Ngưỡng môi trường phải do User tự thiết lập trước khi chế độ tự động được kích hoạt. - Độ sáng thấp hơn ngưỡng thì hệ thống bật đèn; độ sáng vượt ngưỡng thì hệ thống có thể tắt đèn theo cấu hình. - Nhiệt độ hoặc độ ẩm vượt ngưỡng thì hệ thống bật quạt hoặc điều chỉnh quạt lên mức phù hợp. - Hệ thống phải có cơ chế tránh dao động bật/tắt liên tục khi dữ liệu nằm sát ngưỡng. - Mỗi lần kích hoạt tự động đều phải được ghi lại cùng thời gian, giá trị cảm biến và hành động thực hiện.
-----------------------	--

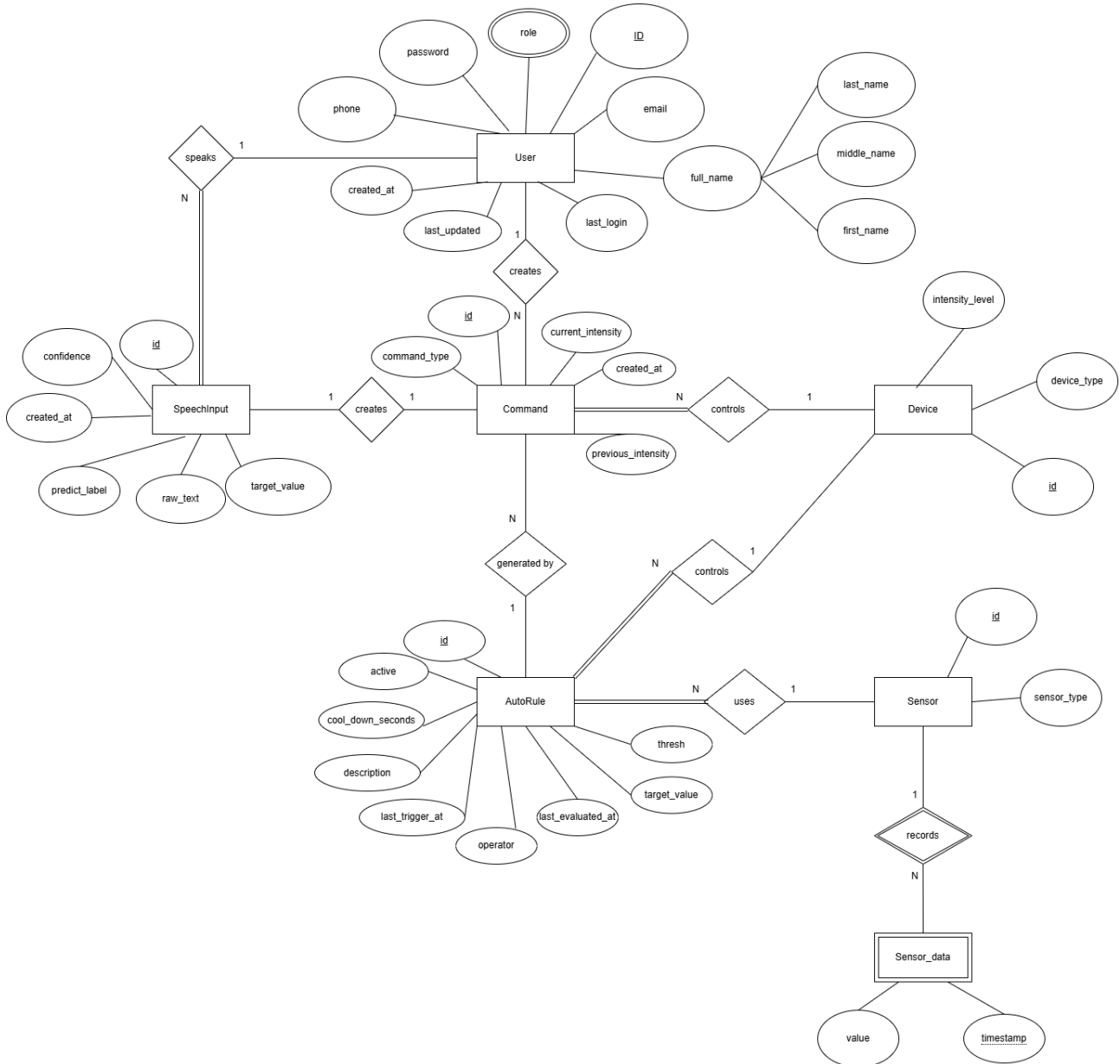
Use-case name:	Xem biểu đồ thống kê
Created by:	Trần Kiến Quốc
Date created:	08/03/2026
Last updated by:	Trần Kiến Quốc
Last updated date:	08/03/2026
Actor:	User, System
Description:	Use case này cho phép người dùng xem biểu đồ thống kê về sự biến thiên của nhiệt độ, độ ẩm và độ sáng theo các mốc thời gian như giờ, ngày và tuần. Chức năng này hỗ trợ người dùng theo dõi điều kiện môi trường và đánh giá hiệu quả vận hành của hệ thống.
Trigger:	Người dùng truy cập màn hình thống kê và yêu cầu xem biểu đồ chi tiết từ dữ liệu môi trường.

Pre-conditions:	- Hệ thống đã thu thập và lưu trữ dữ liệu cảm biến theo thời gian. - Người dùng truy cập được vào Dashboard. - Cơ sở dữ liệu chứa dữ liệu thống kê hợp lệ cho khoảng thời gian được chọn.
Post-conditions:	- Biểu đồ thống kê được hiển thị thành công trên giao diện. - Người dùng có thể quan sát dữ liệu theo mốc thời gian mong muốn. - Hệ thống giữ nguyên dữ liệu và trạng thái hiện tại sau khi hiển thị.
Normal flow:	1. User truy cập vào màn hình thống kê ở giao diện chính. 2. System hiển thị tổng quát về các loại dữ liệu gồm nhiệt độ, độ ẩm, độ sáng và số người ngồi. 3. User chọn loại dữ liệu cần xem. 4. User chọn khoảng thời gian hoặc mốc thống kê, ví dụ: theo 1 giờ, 6 giờ, 24 giờ và 7 ngày. 5. System truy vấn dữ liệu tương ứng từ cơ sở dữ liệu. 6. System xử lý, tổng hợp và chuẩn hóa dữ liệu thống kê. 7. System dựng biểu đồ trực quan từ dữ liệu đã xử lý. 8. System hiển thị biểu đồ lên giao diện cho User theo dõi. 9. User quan sát và phân tích xu hướng biến thiên của dữ liệu môi trường.
Alternative flow:	1. User thay đổi loại dữ liệu cần xem, System tải lại biểu đồ tương ứng.

	2. User thay đổi mốc thời gian, System truy vấn lại dữ liệu theo lựa chọn mới. 3. User có thể xem lần lượt nhiều biểu đồ khác nhau trong cùng một phiên làm việc.
Exception:	- Nếu không có dữ liệu trong khoảng thời gian được chọn, System thông báo không có dữ liệu thống kê. - Nếu truy vấn cơ sở dữ liệu thất bại, System thông báo lỗi và không hiển thị biểu đồ. - Nếu dữ liệu bị thiếu hoặc không hợp lệ, System chỉ hiển thị phần dữ liệu khả dụng hoặc báo lỗi phù hợp.
Business rule:	- Dữ liệu thống kê phải được lấy từ dữ liệu cảm biến đã lưu trữ chính thức trong hệ thống. - Người dùng được phép xem dữ liệu theo các mốc thời gian tối thiểu: giờ, ngày, tuần. - Biểu đồ phải phản ánh đúng thứ tự thời gian và giá trị đo được. - Chỉ dữ liệu hợp lệ mới được sử dụng để hiển thị và phân tích thống kê.

3.2 Thiết kế cơ sở dữ liệu (Database)

3.2.1 Sơ đồ mối quan hệ thực thể (ERD)



Hình 2: Sơ đồ mối quan hệ thực thể (ERD) của hệ thống

Sơ đồ ERD trong Hình 2 mô tả cấu trúc dữ liệu của hệ thống điều khiển thiết bị thông minh thông qua giọng nói. Các thực thể chính bao gồm: *User*, *SpeechInput*, *Command*, *Device*, *Sensor*, *SensorData* và *AutoRule*. Các thực thể này được liên kết với nhau thông qua các mối quan hệ nhằm đảm bảo luồng xử lý từ đầu vào (giọng nói) đến hành động điều khiển thiết bị và tự động hóa.

1. **Thực thể User** lưu trữ thông tin người dùng, bao gồm các thuộc tính như *ID*, *email*, *phone*, *password*, *fullname* (gồm *first_name*, *middle_name*, *last_name*), *created_at*, và *last_login*.
 - Quan hệ **speaks** với thực thể *SpeechInput*: Một người dùng có thể tạo không hoặc nhiều *SpeechInput* (dành cho thao tác điều khiển bằng giọng nói). Và ngược lại, mỗi *SpeechInput* chỉ có thể được tạo bởi một và chỉ một *User*.
 - Quan hệ **creates** với thực thể *Command*: Một người dùng có thể tạo không hoặc nhiều *Command* (dành cho thao tác điều khiển thủ công). Và ngược lại, mỗi *Command* chỉ có thể được tạo bởi một *User*.
2. **Thực thể SpeechInput** biểu diễn dữ liệu đầu vào từ giọng nói, bao gồm các thuộc tính *ID*, *raw_text*. Ngoài ra, các thuộc tính *action_type* và *action_value* được suy diễn từ quá trình xử lý ngôn ngữ tự nhiên.
 - Quan hệ **creates** với thực thể *Command*: Mỗi *SpeechInput* tạo ra một *Command* (đối với trường hợp thành công) hoặc không *Command* (đối với trường hợp thất bại). Và ngược lại, đối với trường hợp điều khiển bằng giọng nói, mỗi *Command* chỉ có thể được tạo bởi một *SpeechInput*.
3. **Thực thể Command** đóng vai trò trung tâm, đại diện cho hành động điều khiển được sinh ra từ giọng nói hoặc luật tự động. Các thuộc tính gồm *ID*, *description*, *action_type*, *action_value*, *timestamp*, *status*.
 - Quan hệ **controls** với thực thể *Device*: Một *Command* có thể điều khiển một và chỉ một *Device*. Ngược lại, mỗi *Device* có thể được điều khiển bởi không hoặc nhiều *Command*.
 - Quan hệ **generated by** với thực thể *AutoRule*: Một *AutoRule* có thể được sinh bởi một *AutoRule*. Ngược lại, mỗi *AutoRule* có thể sinh không hoặc nhiều *Command*.
4. **Thực thể Device** đại diện cho các thiết bị trong hệ thống (đèn, quạt,...), với

các thuộc tính như *device_name*, *device_type*, *power_state*, *intensity_level*, *created_at*, *last_updated*.

- Quan hệ **has** với thực thể *Sensor*: Một thiết bị có thể chứa không hoặc nhiều *Sensor*. Ngược lại, mỗi *Sensor* được tích hợp với không (đối với các cảm biến độc lập như cảm biến nhiệt độ, ánh sáng,...) hoặc 1 (đối với các cảm biến được nhúng vào thiết bị) *Device*.

5. **Thực thể *Sensor*** mô tả các cảm biến gắn trên thiết bị, gồm các thuộc tính *sensor_name*, *sensor_type*, *created_at*, *last_updated*.

- Quan hệ **produces** với thực thể *SensorData*: Một cảm biến tạo ra không hoặc nhiều *SensorData*. Ngược lại, mỗi *SensorData* chỉ thuộc về một và chỉ một *Sensor*.

6. **Thực thể *SensorData*** lưu trữ dữ liệu đo được từ cảm biến, gồm *value* và *timestamp*. Đây là thực thể yếu, phụ thuộc vào *Sensor*.

7. **Thực thể *AutoRule*** biểu diễn các luật tự động hóa trong hệ thống, với các thuộc tính *ID*, *operator*, *threshold_value*, *action_type*, *action_value*, *is_active*, *created_at*, *last_updated*.

- Quan hệ **uses** với thực thể *Sensor*: Một luật có thể sử dụng dữ liệu từ không hoặc nhiều *Sensor* và một cảm biến có thể cung cấp dữ liệu cho không hoặc nhiều *AutoRule*.
- Quan hệ **generated by** với thực thể *Command*: Một luật có thể sinh ra không hoặc nhiều *Command*.

Sơ đồ ERD thể hiện rõ luồng dữ liệu của hệ thống:

- Đối với điều khiển thủ công:

User → Command → Device

- Đối với điều khiển qua giọng nói:

User → SpeechInput → Command → Device

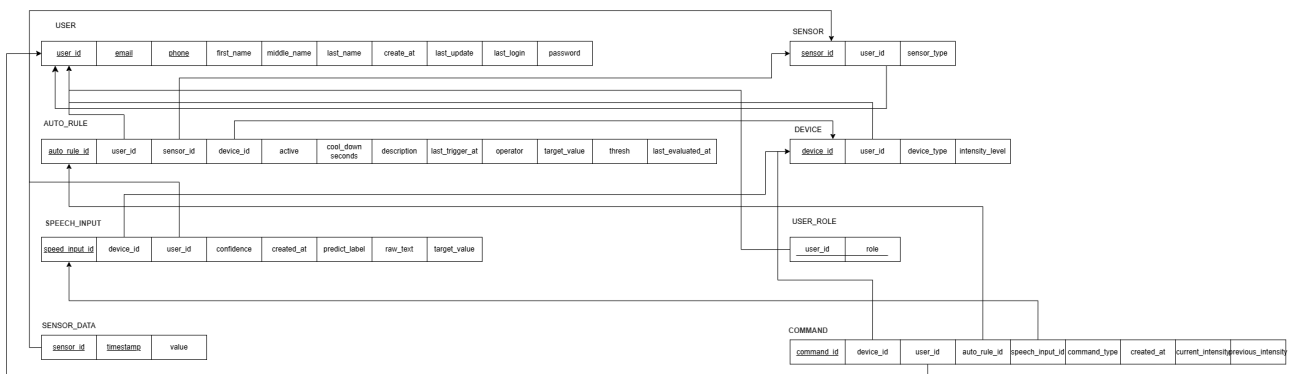
- Đồng thời, hệ thống hỗ trợ tự động hóa thông qua:

Sensor → SensorData → AutoRule → Command

Thiết kế này đảm bảo:

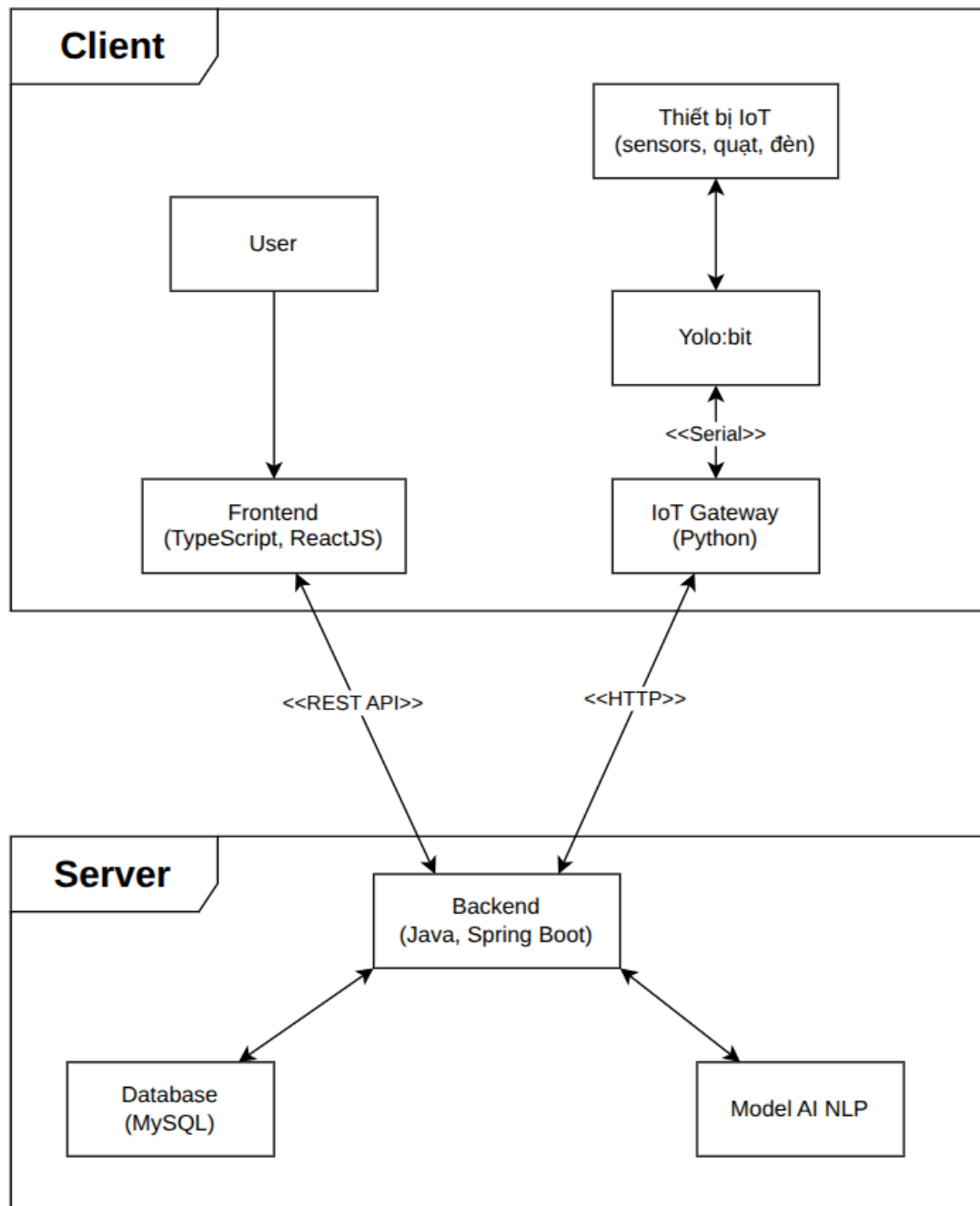
- Tách biệt rõ ràng giữa dữ liệu đầu vào, xử lý và hành động.
- Dễ mở rộng cho các loại thiết bị và cảm biến mới.
- Hỗ trợ cả điều khiển thủ công (giọng nói) và tự động (rule-based).

3.2.2 Lược đồ ánh xạ



Hình 3: Lược đồ ánh xạ

3.3 Kiến trúc hệ thống



Hình 4: Kiến trúc tổng thể của hệ thống phòng học thông minh

Hệ thống "Phòng học thông minh" được thiết kế dựa trên mô hình kiến trúc Client-Server và được chia thành 4 lớp chính:

3.3.1 Lớp thiết bị vật lý

- **Thành phần:** Mạch vi điều khiển trung tâm (YOLO:bit), các cảm biến (nhiệt độ, độ ẩm, độ sáng, cảm biến sóng âm) và các thiết bị điện tử (đèn LED, quạt).
- **Chức năng:** Đóng vai trò thu thập các thông số như nhiệt độ, độ ẩm, ánh sáng và trạng thái hiện diện trong phòng học. Đồng thời thực thi các lệnh bật/tắt thiết bị điện tử.
- **Giao thức:** Giao tiếp nội bộ qua các chân I2C. Dữ liệu sau khi thu thập được truyền lên lớp trên thông qua chuẩn giao tiếp Serial (UART).

3.3.2 Lớp IoT Gateway

- **Thành phần:** Thiết bị IoT Gateway (Laptop) chạy mã nguồn Python.
- **Chức năng:** Đây là cầu nối của hệ thống. Gateway thực hiện chức năng phân tích luồng dữ liệu Serial thô từ vi điều khiển, đóng gói thành định dạng chuẩn (JSON) và gửi lên Backend và theo chiều ngược lại.
- **Giao thức:** Giao tiếp với Backend thông qua giao thức mạng HTTP.

3.3.3 Lớp Server (Backend, Database)

- **Thành phần:** Máy chủ Backend (Java, Spring Boot) và Cơ sở dữ liệu (MySQL).
- **Chức năng:** Đóng vai trò lưu trữ và điều phối. Lớp này cung cấp các RESTful API để nhận dữ liệu từ Gateway, xử lý các logic nghiệp vụ phức tạp (như so sánh dữ liệu cảm biến với ngưỡng cài đặt để kích hoạt hệ thống tự động), gửi dữ liệu điều khiển đến Gateway và lưu trữ lịch sử hoạt động vào Database.

3.3.4 Lớp giao diện người dùng (Frontend)

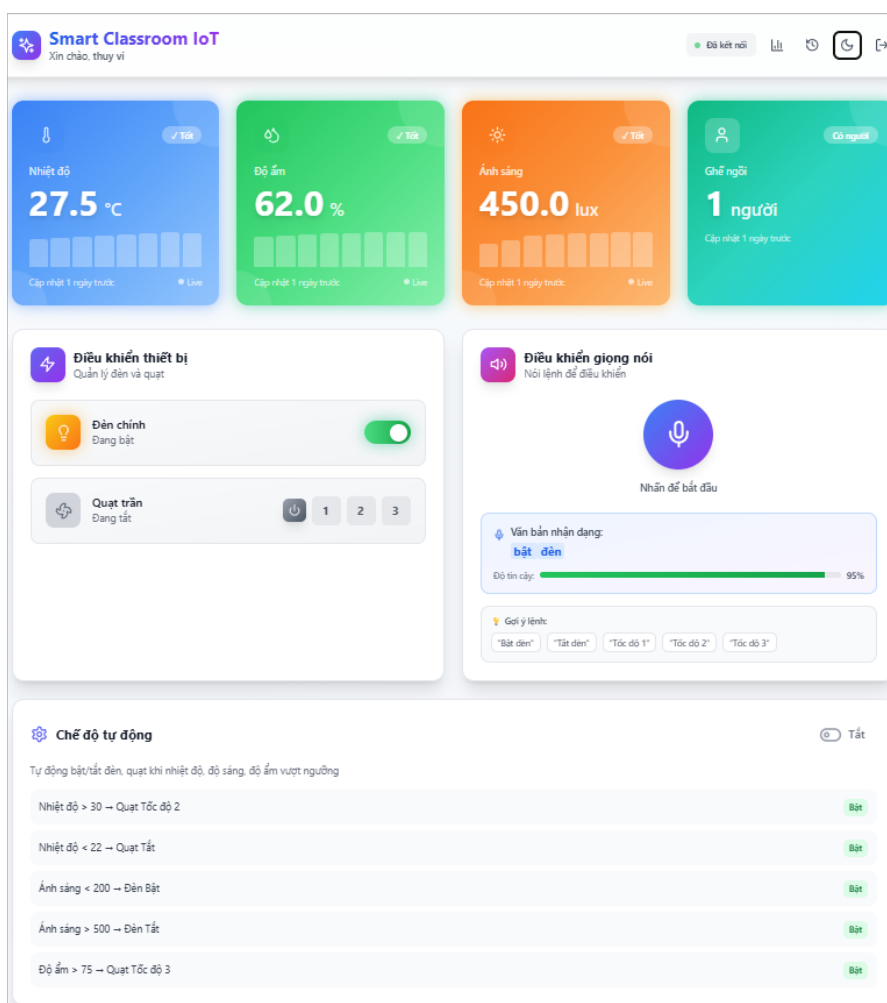
- **Thành phần:** Ứng dụng Web Dashboard (TypeScript, ReactJS).

- **Chức năng:** Giao tiếp trực tiếp với User. Trực quan hóa dữ liệu môi trường bằng các biểu đồ tương tác, hiển thị lịch sử lệnh và cung cấp các nút bấm để ghi đè lệnh điều khiển thủ công (Manual Override).
- **Giao thức:** Sử dụng các REST API từ lớp máy chủ thông qua các HTTP Requests.

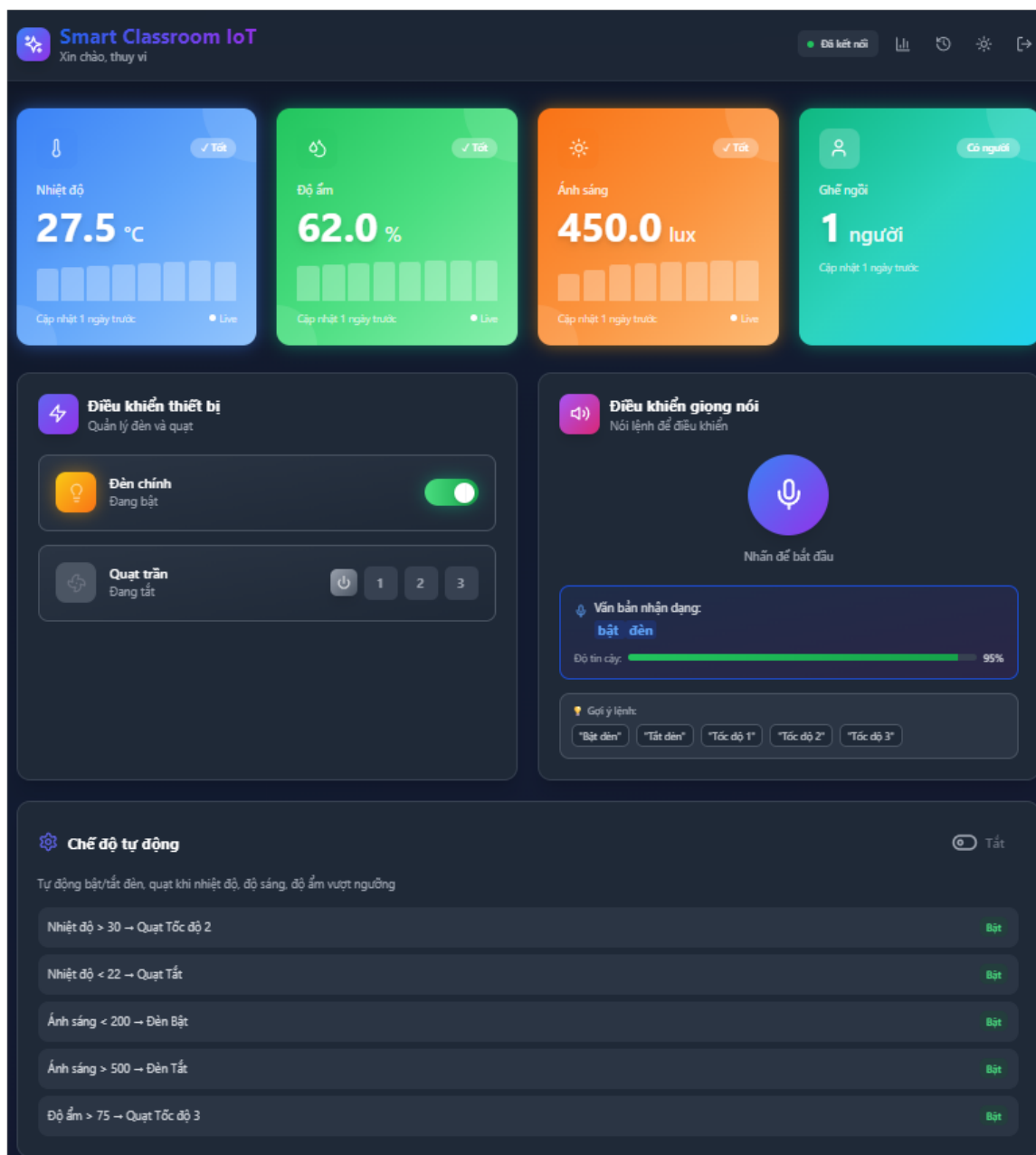
3.4 Thiết kế giao diện

Phần giao diện người dùng của hệ thống được triển khai dưới dạng Web Dashboard.

3.4.1 Màn hình trang chủ



Hình 5: Giao diện trang chủ ở chế độ sáng



Hình 6: Giao diện trang chủ ở chế độ tối

3.4.2 Màn hình lịch sử và thống kê



Hình 7: Giao diện lịch sử hoạt động tổng quan

← **Lịch sử hoạt động** [Xuất CSV](#)

Bộ lọc

Nguồn kích hoạt: Tất cả Trạng thái: Tất cả

THỜI GIAN	THIẾT BỊ	HÀNH ĐỘNG	TRẠNG THÁI	NGUỒN
14/03/2026 20:16:14	Đèn chính	Bật	Thất bại	Thủ công
14/03/2026 20:16:14	Quạt trần	Tắt	Thành công	Tự động
14/03/2026 20:16:14	AC Unit	Đặt tốc độ 2	Thành công	Giọng nói
14/03/2026 20:16:14	Đèn chính	Đặt tốc độ 3	Thành công	Thủ công
13/03/2026 20:16:14	Quạt trần	Bật	Thành công	Tự động
13/03/2026 20:16:14	AC Unit	Tắt	Thất bại	Giọng nói
13/03/2026 20:16:14	Đèn chính	Đặt tốc độ 2	Thành công	Thủ công
13/03/2026 20:16:14	Quạt trần	Đặt tốc độ 3	Thành công	Tự động
12/03/2026 20:16:14	AC Unit	Bật	Thành công	Giọng nói
12/03/2026 20:16:14	Đèn chính	Tắt	Thành công	Thủ công
12/03/2026 20:16:14	Quạt trần	Đặt tốc độ 2	Thất bại	Tự động
12/03/2026 20:16:14	AC Unit	Đặt tốc độ 3	Thành công	Giọng nói
11/03/2026 20:16:14	Đèn chính	Bật	Thành công	Thủ công
11/03/2026 20:16:14	Quạt trần	Tắt	Thành công	Tự động
11/03/2026 20:16:14	AC Unit	Đặt tốc độ 2	Thành công	Giọng nói

Hình 8: Giao diện lịch sử lệnh điều khiển chi tiết

4 Hiện thực hệ thống

4.1 Mô hình Xử lý ngôn ngữ tự nhiên (NLP)

Trong hệ thống phòng học thông minh, mô hình xử lý ngôn ngữ tự nhiên được xây dựng nhằm nhận diện ý định điều khiển thiết bị từ câu lệnh tiếng Việt của người dùng. Người dùng có thể nhập hoặc nói các câu lệnh tự nhiên như “mở quạt”, “tắt đèn”, “phòng tối quá” hoặc “cho quạt mạnh hơn”. Sau khi tiếp nhận câu lệnh, hệ thống sẽ tiến hành xử lý văn bản, trích xuất đặc trưng, dự đoán ý định và trả về nhãn điều khiển tương ứng cho backend.

Bài toán NLP trong hệ thống được xem như một bài toán phân loại văn bản ngắn. Đầu vào của mô hình là một câu lệnh tiếng Việt, đầu ra là nhãn biểu diễn ý định điều khiển thiết bị. Trong phiên bản hiện thực, mô hình tập trung vào các nhóm lệnh chính liên quan đến quạt và đèn, bao gồm bật quạt, tắt quạt, bật đèn và tắt đèn. Các nhãn này sau đó được ánh xạ sang các lệnh hệ thống như `TURN_ON_FAN`, `TURN_OFF_FAN`, `TURN_ON_LIGHT` và `TURN_OFF_LIGHT`.

Quy trình xây dựng mô hình bao gồm các bước chính: chuẩn bị tập dữ liệu, tiền xử lý văn bản tiếng Việt, trích xuất đặc trưng bằng TF-IDF, lựa chọn đặc trưng bằng phương pháp Chi-Square, huấn luyện mô hình Logistic Regression và đánh giá mô hình trên tập kiểm thử.

4.1.1 Tập dữ liệu

Tập dữ liệu được xây dựng từ các câu lệnh tiếng Việt mô phỏng tình huống người dùng điều khiển thiết bị trong phòng học thông minh. Dữ liệu bao gồm các mẫu câu có nhiều cách diễn đạt khác nhau nhằm tăng khả năng nhận diện của mô hình đối với ngôn ngữ tự nhiên. Ví dụ, cùng một ý định bật quạt có thể được diễn đạt bằng nhiều câu như “mở quạt”, “bật quạt lên”, “phòng hơi bí, cho quạt chạy đi” hoặc “giúp mình mở quạt để dễ chịu hơn”.

Trong quá trình thử nghiệm, hệ thống sử dụng tập dữ liệu gồm 2400 mẫu với hai trường chính: câu lệnh đầu vào và nhãn tương ứng. Tập dữ liệu được chia thành bốn lớp

chính: `bat_quat`, `tat_quat`, `bat_den` và `tat_den`. Mỗi lớp có 600 mẫu, do đó dữ liệu có phân bố cân bằng giữa các nhãn. Việc dữ liệu cân bằng giúp quá trình huấn luyện ổn định hơn và làm cho các chỉ số đánh giá như accuracy, precision, recall và F1-score phản ánh khách quan hơn hiệu năng của mô hình.

Trước khi huấn luyện, dữ liệu được kiểm tra về giá trị thiếu, câu rỗng, độ dài câu và các ký tự đặc biệt. Kết quả cho thấy dữ liệu không chứa giá trị thiếu và không có câu rỗng. Phần lớn các câu lệnh là câu ngắn, có độ dài trung bình khoảng 8 từ, phù hợp với đặc trưng của bài toán điều khiển thiết bị bằng giọng nói hoặc câu lệnh nhanh. Một số câu có chứa dấu câu hoặc ký tự đặc biệt, tuy nhiên các ký tự này chủ yếu là dấu câu thông thường và không mang nhiều ý nghĩa phân biệt nhãn, vì vậy được loại bỏ trong bước tiền xử lý.

Do tiếng Việt là ngôn ngữ đa âm tiết, một từ có thể gồm nhiều tiếng được ngăn cách bởi khoảng trắng. Vì vậy, nếu chỉ tách từ dựa trên dấu cách thì có thể làm mất ý nghĩa của các cụm từ. Để khắc phục vấn đề này, hệ thống sử dụng thư viện `underthesea` để tách từ tiếng Việt trước khi đưa dữ liệu vào mô hình.

4.1.2 Huấn luyện mô hình

Quá trình huấn luyện mô hình bắt đầu bằng bước tiền xử lý văn bản. Mỗi câu lệnh đầu vào được chuẩn hóa theo một pipeline thống nhất gồm các bước: chuyển toàn bộ văn bản về chữ thường, loại bỏ khoảng trắng dư thừa, loại bỏ dấu câu và ký tự đặc biệt, chuẩn hóa một số từ viết tắt hoặc từ lóng, sau đó thực hiện tách từ tiếng Việt bằng `underthesea`. Ví dụ, các từ như “ko”, “k”, “hok” được chuẩn hóa thành “không”, còn “giùm” hoặc “dum” được chuẩn hóa thành “giúp”. Việc chuẩn hóa này giúp giảm nhiễu trong dữ liệu và làm cho các câu có cùng ý nghĩa được biểu diễn nhất quán hơn.

Sau bước tiền xử lý, văn bản được chuyển sang dạng vector số bằng phương pháp TF-IDF. Phương pháp này biểu diễn văn bản dựa trên mức độ quan trọng của từ hoặc cụm từ trong từng câu so với toàn bộ tập dữ liệu. Hệ thống sử dụng cả unigram và bigram để mô hình có thể học được không chỉ các từ đơn như “bật”, “tắt”, “quạt”, “đèn”, mà còn cả các cụm từ có ý nghĩa như “bật quạt”, “tắt đèn”, “sáng quá” hoặc “tối quá”.

Do số lượng đặc trưng sinh ra từ TF-IDF có thể lớn và chứa nhiều đặc trưng ít

đóng góp cho việc phân loại, hệ thống áp dụng phương pháp chọn đặc trưng Chi-Square. Phương pháp này giúp lựa chọn các đặc trưng có mối liên hệ mạnh với nhãn đầu ra, từ đó giảm nhiễu, giảm số chiều dữ liệu và cải thiện khả năng tổng quát hóa của mô hình.

Mô hình phân loại được sử dụng là Logistic Regression. Đây là mô hình tuyến tính phù hợp với bài toán phân loại văn bản ngắn, đặc biệt khi dữ liệu đã được biểu diễn dưới dạng vector TF-IDF. Logistic Regression có ưu điểm là tốc độ huấn luyện nhanh, dễ triển khai, hiệu quả với dữ liệu sparse và cho phép tính xác suất dự đoán để đánh giá độ tin cậy của kết quả.

Để tìm cấu hình mô hình phù hợp, quá trình tối ưu siêu tham số được thực hiện qua hai giai đoạn. Giai đoạn đầu sử dụng RandomizedSearchCV để khảo sát nhanh không gian tham số, bao gồm hệ số điều chuẩn `C`, loại penalty, solver và class weight. Sau đó, GridSearchCV được sử dụng để tinh chỉnh chi tiết trong vùng tham số tốt nhất. Kết quả tối ưu cho mô hình là Logistic Regression với `C = 40`, `penalty = l2`, `solver = lbfgs` và `class_weight = None`.

Sau khi huấn luyện, các thành phần của mô hình bao gồm `vectorizer`, `selector` và mô hình phân loại được lưu lại dưới dạng các file `.pkl`. Các file này được nạp lại trong AI service để phục vụ dự đoán trong quá trình hệ thống hoạt động.

4.1.3 Thử nghiệm và đánh giá

Mô hình được đánh giá trên tập kiểm thử tách riêng từ tập dữ liệu ban đầu. Các chỉ số được sử dụng để đánh giá bao gồm accuracy, precision, recall, F1-score và confusion matrix. Đây là các chỉ số phổ biến trong bài toán phân loại, giúp đánh giá không chỉ độ chính xác tổng thể mà còn hiệu quả nhận diện trên từng lớp nhãn.

Kết quả thử nghiệm cho thấy mô hình đạt độ chính xác rất cao trên tập kiểm thử. Cụ thể, accuracy đạt 1.0, đồng thời precision, recall và F1-score của cả bốn lớp `bat_quat`, `tat_quat`, `bat_den` và `tat_den` đều đạt 1.0. Ma trận nhầm lẫn cho thấy toàn bộ mẫu kiểm thử được phân loại đúng, không xuất hiện trường hợp nhầm lẫn giữa các lớp.

```

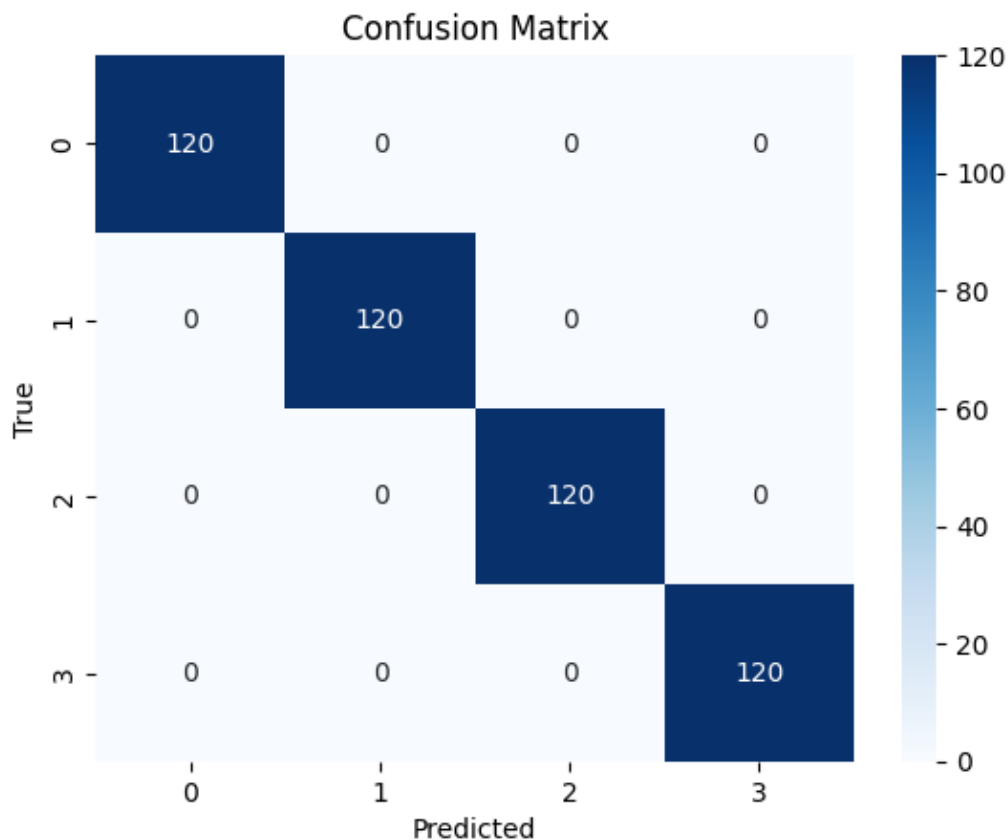
Accuracy: 1.0

Classification Report:

```

	precision	recall	f1-score	support
bat_den	1.00	1.00	1.00	120
bat_quat	1.00	1.00	1.00	120
tat_den	1.00	1.00	1.00	120
tat_quat	1.00	1.00	1.00	120
accuracy			1.00	480
macro avg	1.00	1.00	1.00	480
weighted avg	1.00	1.00	1.00	480

Hình 9: Đánh giá mô hình



Hình 10: Ma trận nhầm lẫn

Kết quả này cho thấy pipeline gồm tiền xử lý tiếng Việt, TF-IDF, lựa chọn đặc trưng Chi-Square và Logistic Regression hoạt động hiệu quả đối với tập dữ liệu hiện

tại. Các câu lệnh trong tập dữ liệu có đặc trưng phân biệt rõ ràng, ví dụ các từ khóa như “bật”, “tắt”, “quạt”, “đèn”, “sáng”, “tối” đóng vai trò quan trọng trong việc xác định nhân.

Tuy nhiên, kết quả đạt độ chính xác tuyệt đối cũng cần được xem xét một cách thận trọng. Nguyên nhân có thể đến từ việc dữ liệu thử nghiệm còn tương đối sạch, các mẫu câu giữa tập huấn luyện và tập kiểm thử có phân phối gần giống nhau, đồng thời số lượng ý định chưa quá phức tạp. Khi triển khai thực tế, người dùng có thể nhập các câu mơ hồ, câu sai chính tả, câu có nhiều ý định cùng lúc hoặc câu không liên quan đến điều khiển thiết bị. Vì vậy, hệ thống bổ sung cơ chế kiểm tra độ tin cậy bằng xác suất dự đoán của mô hình. Nếu độ tin cậy nhỏ hơn 0.7, kết quả sẽ được gán nhãn UNKNOWN để tránh thực thi nhầm lệnh.

Trong quá trình triển khai, mô hình NLP được đóng gói thành một dịch vụ riêng bằng FastAPI. Backend gửi câu lệnh người dùng đến endpoint `/predict`; AI service sẽ tiền xử lý câu lệnh, vector hóa bằng TF-IDF, chọn đặc trưng, dự đoán nhãn và trả về kết quả gồm `predictLabel` và `confidence`. Cách thiết kế này giúp tách riêng phần xử lý AI khỏi backend chính, thuận tiện cho việc bảo trì, nâng cấp mô hình hoặc thay thế bằng mô hình NLP phức tạp hơn trong tương lai.

Nhìn chung, mô hình NLP đã đáp ứng tốt yêu cầu nhận diện ý định điều khiển thiết bị trong phạm vi bài toán của hệ thống. Mô hình có ưu điểm là đơn giản, tốc độ xử lý nhanh, dễ triển khai và phù hợp với dữ liệu câu lệnh ngắn tiếng Việt. Trong các phiên bản tiếp theo, hệ thống có thể được cải tiến bằng cách mở rộng tập dữ liệu thực tế, bổ sung thêm các nhãn như tăng/giảm cường độ thiết bị, xử lý câu lệnh đa ý định và thử nghiệm các mô hình học sâu hoặc mô hình ngôn ngữ tiếng Việt để tăng khả năng hiểu ngữ cảnh.

4.2 Iot Gateway

Thành phần IoT Gateway đóng vai trò là trạm trung chuyển trung tâm, chịu trách nhiệm kết nối giao tiếp giữa lớp thiết bị phần cứng (Yolo:Bit) và máy chủ Backend (Java + Spring Boot). Gateway được phát triển bằng ngôn ngữ Python, sử dụng thư viện `serial` để giao tiếp qua cổng COM và thư viện `requests` để truyền nhận dữ liệu qua

giao thức HTTP RESTful API.

4.2.1 Hàm tự động phát hiện và kết nối thiết bị

Để tăng tính linh hoạt và trải nghiệm người dùng, hệ thống không gán cứng tên cổng COM mà triển khai cơ chế tự động dò tìm thông qua hàm `get_port()`.

- **Quét thiết bị ngoại vi:** Hệ thống sử dụng thư viện `serial.tools.list_ports` để liệt kê toàn bộ các cổng giao tiếp đang kết nối với máy tính.
- **Định danh thiết bị:** Gateway tiến hành duyệt qua danh sách và lọc ra thiết bị có chứa từ khóa đặc trưng của mạch nạp vi điều khiển là "CH340" hoặc "USB".
- **Thiết lập kết nối:** Sau khi xác định đúng cổng (ví dụ: COM3), hệ thống khởi tạo kết nối Serial với cấu hình tốc độ baud là 115200 và thời gian chờ (timeout) là 0.1s. Quá trình này giúp Gateway duy trì giao tiếp ổn định với Yolo:Bit và dễ dàng phục hồi ném ngoại lệ nếu cáp kết nối bị lỏng hoặc ngắt đột ngột.

4.2.2 Xử lý dữ liệu chiều lên (từ Yolo:Bit đến Server)

Nhiệm vụ này chịu trách nhiệm thu thập dữ liệu môi trường thô từ các cảm biến vật lý và đẩy lên máy chủ.

- **Đọc dữ liệu không đồng bộ:** Hệ thống liên tục kiểm tra bộ đệm Serial thông qua thuộc tính `ser.in_waiting > 0`. Cách tiếp cận này giúp Gateway chỉ đọc khi thực sự có dữ liệu, tránh tình trạng treo (blocking) toàn bộ chương trình.
- **Phân tích cú pháp:** Dữ liệu chuỗi (chuẩn UTF-8) gửi từ Yolo:Bit được nhận diện qua các tiền tố cụ thể:
 - **"T:"**: Cắt bỏ tiền tố để lấy giá trị số thực của Nhiệt độ (bbc-temp).
 - **"H:"**: Lấy giá trị Độ ẩm (bbc-humi).
 - **"L:"**: Lấy giá trị cường độ Ánh sáng (bbc-lux).

- **Đóng gói và truyền tải:** Sau khi bóc tách, dữ liệu được gom thành một đối tượng JSON và gửi thẳng lên endpoint `/api/gateway/sensor-data` của Backend bằng phương thức HTTP POST.

4.2.3 Xử lý dữ liệu chiều xuống (từ Server đến Yolo:Bit)

Để thực thi các lệnh điều khiển đèn và quạt từ giao diện Web Dashboard hoặc mô hình AI nhận diện giọng nói, Gateway thực hiện cơ chế lấy dữ liệu (polling) từ máy chủ để truyền xuống phân cứng.

- **Cơ chế Polling:** Sử dụng hàm `time.time()`, Gateway tự động gửi một HTTP GET request đến endpoint `/api/gateway/control-state` theo chu kỳ cố định là **1.0 giây/lần**.
- **Lọc nhiễu và tối ưu đường truyền:** Nhằm tránh việc gửi lệnh dư thừa xuống vi điều khiển gây quá tải hoặc xung đột, hệ thống lưu lại trạng thái liền trước (`last_light_state` và `last_fan_state`). Gateway chỉ phát lệnh qua Serial `ser.write()` khi phát hiện có sự thay đổi (sai khác) giữa trạng thái mới nhận từ Backend và trạng thái cũ.
- **Quy chuẩn lệnh giao tiếp:**
 - **Đối với đèn:** Lệnh bật là chuỗi `b"1\n"` và tắt là `b"0\n"`.
 - **Đối với quạt (PWM):** Hệ thống quy đổi 3 mức tốc độ từ Web thành các giá trị phần trăm (Duty Cycle). Cụ thể: mức 0 (tắt) tương ứng `b"S0\n"`, các mức 1, 2, 3 được nội suy tương ứng với các giá trị xung S33, S66 và S100 để điều khiển tốc độ động cơ DC.

4.2.4 Cơ chế điều phối tác vụ không đồng bộ

IoT Gateway sử dụng mô hình đa luồng để xử lý đồng thời hai luồng dữ liệu độc lập. Thread thứ nhất chạy hàm `serial_reader()`, liên tục đọc dữ liệu cảm biến từ cổng Serial, phân tích các tiền tố T:, H:, L: và gửi dữ liệu lên Backend thông qua HTTP POST tại endpoint `/iot/sensor-data`. Thread thứ hai chạy hàm `start_ws()`,

duy trì kết nối WebSocket với Backend, sử dụng giao thức STOMP để subscribe topic `/topic/commands` và nhận lệnh điều khiển thiết bị.

Khi Backend gửi lệnh điều khiển, Gateway xử lý message trong callback `on_message()`, ánh xạ lệnh theo loại thiết bị. Với quạt, lệnh được chuyển thành dạng `S<value>`; với đèn, giá trị lớn hơn 0 được chuyển thành 1, ngược lại là 0. Sau đó Gateway ghi lệnh xuống thiết bị thông qua `ser.write()`.

Cách tổ chức này giúp Gateway vừa đọc dữ liệu cảm biến theo chiều thiết bị lên Backend, vừa nhận lệnh điều khiển theo chiều Backend xuống thiết bị mà không phải chờ tuần tự trong cùng một luồng. Nếu kết nối WebSocket bị ngắt, Gateway tự tạo lại kết nối sau khoảng 2 giây.

5 Thiết lập và khởi chạy dự án

Mã nguồn được tổ chức theo kiến trúc nhiều thành phần. Backend Spring Boot đóng vai trò API trung tâm, lưu trữ dữ liệu vào MySQL, xử lý xác thực, điều khiển thiết bị, luật tự động và lệnh giọng nói. Front-end React/Vite cung cấp giao diện người dùng để theo dõi cảm biến, điều khiển thiết bị và xem lịch sử. Phần IoT Gateway kết nối với Yolo:Bit qua cổng serial, gửi dữ liệu cảm biến lên backend và nhận lệnh điều khiển từ backend qua WebSocket/STOMP. Ngoài ra, hệ thống có một AI service riêng dùng FastAPI để phân loại câu lệnh tiếng Việt thành intent điều khiển.

5.1 Backend

Backend được đặt trong thư mục `backend/` và được xây dựng bằng Spring Boot, Maven và Java. Cấu trúc mã nguồn tuân theo mô hình phân tầng gồm `controller`, `service`, `repository`, `entity`, `dto`, `mapper` và `configuration`. Các controller nhận request REST, service xử lý nghiệp vụ, repository thao tác với MySQL thông qua Spring Data JPA, còn mapper dùng MapStruct để chuyển đổi giữa entity và DTO.

Trước khi chạy backend cần chuẩn bị MySQL và tạo cơ sở dữ liệu:

```
CREATE DATABASE smart_study_room;
```

Các biến môi trường cần thiết cho backend gồm:

```
SPRING_DATASOURCE_URL=jdbc:mysql://localhost:3306/smart_study_room
```

```
SPRING_DATASOURCE_USERNAME=root
```

```
SPRING_DATASOURCE_PASSWORD=<mat_khau_mysql>
```

```
JWT_SIGNER_KEY=<chuoi_jwt>
```

```
AI_SERVICE_URL=http://localhost:8000
```

```
CORS_ALLOWED_ORIGINS=http://localhost:3000,http://127.0.0.1:3000
```

Sau khi cấu hình, backend có thể được khởi chạy bằng Maven:

```
cd backend
```

```
./mvnw spring-boot:run
```


Trên Windows có thể dùng:

```
cd backend  
mvnw.cmd spring-boot:run
```

Backend mặc định chạy tại địa chỉ:

`http://localhost:8080`

Khi khởi động, backend tự động tạo tài khoản quản trị và một tài khoản kiểm thử cố định. Với mỗi người dùng mới, hệ thống cũng khởi tạo mặc định hai thiết bị là **FAN**, **LIGHT** và ba cảm biến là **TEMPERATURE**, **HUMIDITY**, **LIGHT**. Backend bảo vệ các API bằng JWT, đồng thời mở public endpoint `/iot/sensor-data` để gateway gửi dữ liệu cảm biến lên hệ thống. Các lệnh điều khiển thiết bị được backend phát qua WebSocket topic `/topic/commands` để gateway nhận và chuyển tiếp xuống phần cứng.

Nếu chạy bằng Docker, thư mục backend có sẵn `docker-compose.yaml` để khởi tạo backend và MySQL. Khi dùng cách này cần bảo đảm biến JWT được đặt đúng tên `JWT_SIGNER_KEY` theo cấu hình trong `application.yaml`.

5.2 Iot Gateway và Yolo:Bit

Phần IoT Edge nằm trong thư mục `iot-edge/`. Thành phần này gồm hai phần chính: chương trình `sensor_node.py` chạy trên Yolo:Bit và chương trình `gateway.py` chạy trên máy tính kết nối với Yolo:Bit qua serial.

Trên Yolo:Bit, chương trình đọc dữ liệu từ cảm biến DHT20 và cảm biến ánh sáng, sau đó gửi dữ liệu qua serial theo định dạng:

```
T:<temperature>  
H:<humidity>  
L:<light>
```

Gateway đọc các dòng serial này, phân tích thành loại cảm biến tương ứng rồi gửi lên backend qua endpoint:

POST `http://localhost:8080/iot/sensor-data`

Ở chiều điều khiển ngược lại, gateway kết nối WebSocket/STOMP tới:

`ws://localhost:8080/ws`

và subscribe topic:

`/topic/commands`

Khi backend phát lệnh điều khiển, gateway chuyển đổi lệnh thành dữ liệu serial để gửi xuống Yolo:Bit. Ví dụ, lệnh điều khiển quạt được map thành `S<percent>`, còn đèn được map thành 1 hoặc 0.

Trước khi chạy gateway cần cài các thư viện Python:

```
cd iot-edge
python -m pip uninstall serial -y
python -m pip install -r requirements.txt
```

Các biến môi trường thường dùng:

```
SMART_ROOM_USER_ID=c7ab5c64-cee4-4ef6-9b2e-1f71824c0920
SMART_ROOM_SERIAL_PORT=COM3
SMART_ROOM_BAUDRATE=115200
SMART_ROOM_BACKEND_URL=http://localhost:8080
SMART_ROOM_WS_URL=ws://localhost:8080/ws
```

Sau đó chạy gateway:

```
python gateway.py
```

Trong trường hợp chưa có thiết bị thật, dự án cung cấp hai script kiểm thử. Script `test_sensor_flow.py` sinh dữ liệu cảm biến giả và gửi định kỳ lên backend. Script `test_device_control_flow.py` lắng nghe lệnh WebSocket từ backend và in trạng thái thiết bị ra terminal. Hai script này giúp kiểm thử luồng cảm biến và luồng điều khiển mà không cần kết nối Yolo:Bit thật.

5.3 AI Service

AI service nằm trong thư mục `ai-service/` và được dùng cho chức năng điều khiển giọng nói. Service này chạy bằng FastAPI, nhận câu lệnh tiếng Việt tại endpoint `/predict`, tiền xử lý văn bản, vector hóa bằng TF-IDF, chọn đặc trưng và dùng mô hình đã huấn luyện để dự đoán intent như `TURN_ON_FAN`, `TURN_OFF_FAN`, `TURN_ON_LIGHT`, `TURN_OFF_LIGHT`. Để chạy service:

```
cd ai-service
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
uvicorn service:app --host 0.0.0.0 --port 8000
```

Trên Windows, lệnh kích hoạt môi trường ảo là:

```
.\venv\Scripts\activate
```

5.4 Front-end

Front-end nằm trong thư mục `front-end/` và được xây dựng bằng React, TypeScript, Vite, Tailwind CSS, React Router và TanStack Query. Cấu trúc mã nguồn gồm các thư mục `pages`, `components`, `hooks`, `contexts`, `services`, `types` và `utils`. Các trang chính gồm dashboard, biểu đồ, lịch sử, cấu hình auto-rule, hồ sơ người dùng, lịch sử giọng nói và trang quản trị.

Front-end mặc định gọi backend qua biến:

```
VITE_API_BASE_URL=http://localhost:8080
```

Có thể tạo file `.env.local` trong thư mục `front-end/` để cấu hình:

```
VITE_API_BASE_URL=http://localhost:8080
```

```
VITE_WS_URL=ws://localhost:8080/ws
```

Sau đó cài dependency và chạy giao diện:

```
cd front-end
```

```
npm install
```

```
npm run dev
```

Ứng dụng mặc định chạy tại:

`http://localhost:3000`

Front-end sử dụng JWT lưu trong local storage để gọi các API cần xác thực. Lớp `services/api.ts` chịu trách nhiệm cấu hình Axios, tự động gắn token vào request, xử lý lỗi xác thực và cung cấp các hàm gọi API cho đăng nhập, đăng ký, cảm biến, thiết bị, auto-rule, lịch sử và giọng nói. Dashboard lấy dữ liệu cảm biến mới nhất từ backend; nếu cấu hình Adafruit IO được bật thông qua `VITE_AIO_USERNAME` và `VITE_AIO_KEY`, front-end có thể dùng adapter Adafruit IO để lấy dữ liệu feed. Khi build production, dùng lệnh:

```
npm run build
```

6 Tổng kết

Đề tài đã trình bày quá trình phân tích, thiết kế và xây dựng hệ thống “Phòng học cá nhân thông minh” nhằm hỗ trợ người dùng giám sát môi trường học tập và điều khiển các thiết bị trong phòng một cách linh hoạt. Hệ thống kết hợp các cảm biến nhiệt độ, độ ẩm, ánh sáng và hiện diện với bộ xử lý trung tâm, máy chủ Backend, cơ sở dữ liệu và giao diện Web Dashboard. Thông qua đó, người dùng có thể theo dõi dữ liệu theo thời gian thực, điều khiển đèn/quạt thủ công hoặc bằng giọng nói, đồng thời sử dụng cơ chế tự động hóa theo ngưỡng để nâng cao sự tiện nghi trong quá trình học tập.

Nhìn chung, hệ thống đáp ứng được các chức năng chính đã đề ra, bao gồm thu thập dữ liệu cảm biến, điều khiển thiết bị, tự động hóa và thống kê lịch sử hoạt động. Bên cạnh kết quả đạt được, hệ thống vẫn có thể được mở rộng thêm trong tương lai như cải thiện độ chính xác của xử lý giọng nói, tối ưu giao diện người dùng, bổ sung các cảm biến mới và tăng cường khả năng phân tích dữ liệu để đưa ra gợi ý thông minh



hơn. Đây là nền tảng phù hợp để phát triển các ứng dụng IoT phục vụ môi trường học tập cá nhân hóa.

Mã nguồn của đề tài được lưu trữ tại GitHub: <https://github.com/buinhatqui/SmartStudyRoom>.

Tài liệu tham khảo

- [1] The Dariu Foundation, *Phát triển IoT Gateway bằng Python*. Vietnam: The Dariu Foundation, 2021, [Online].
- [2] OhStem Education, *Kết nối vạn vật với Yolo:Bit (Yolo:Farm AIoT)*. Vietnam: OhStem Education, 2021, [Online].
- [3] —, *Kết nối vạn vật với Yolo:Bit*. Vietnam: OhStem Education, 2021.
- [4] Viblo, “Logistic regression – bài toán cơ bản trong machine learning,” <https://viblo.asia/p/logistic-regression-bai-toan-co-ban-trong-machine-learning-924lJ4rzKPM>.